Convolutional Neural Network

Edges

- **Edge:** are abrupt changes in intensity, discontinuity in image brightness or contrast; usually edges occur on the boundary of two regions.

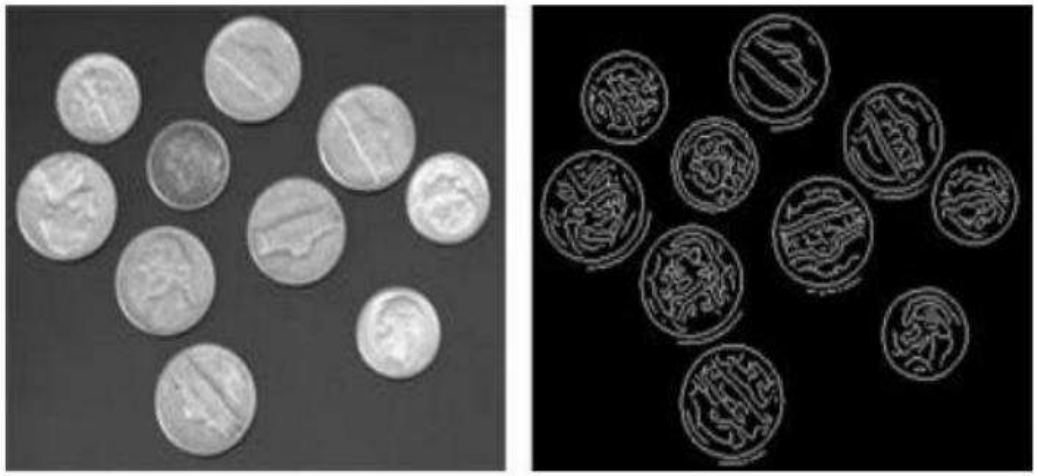


Figure: Original image (left) and edge (right)

Edges

Vertical edges

Horizontal edges

Edges

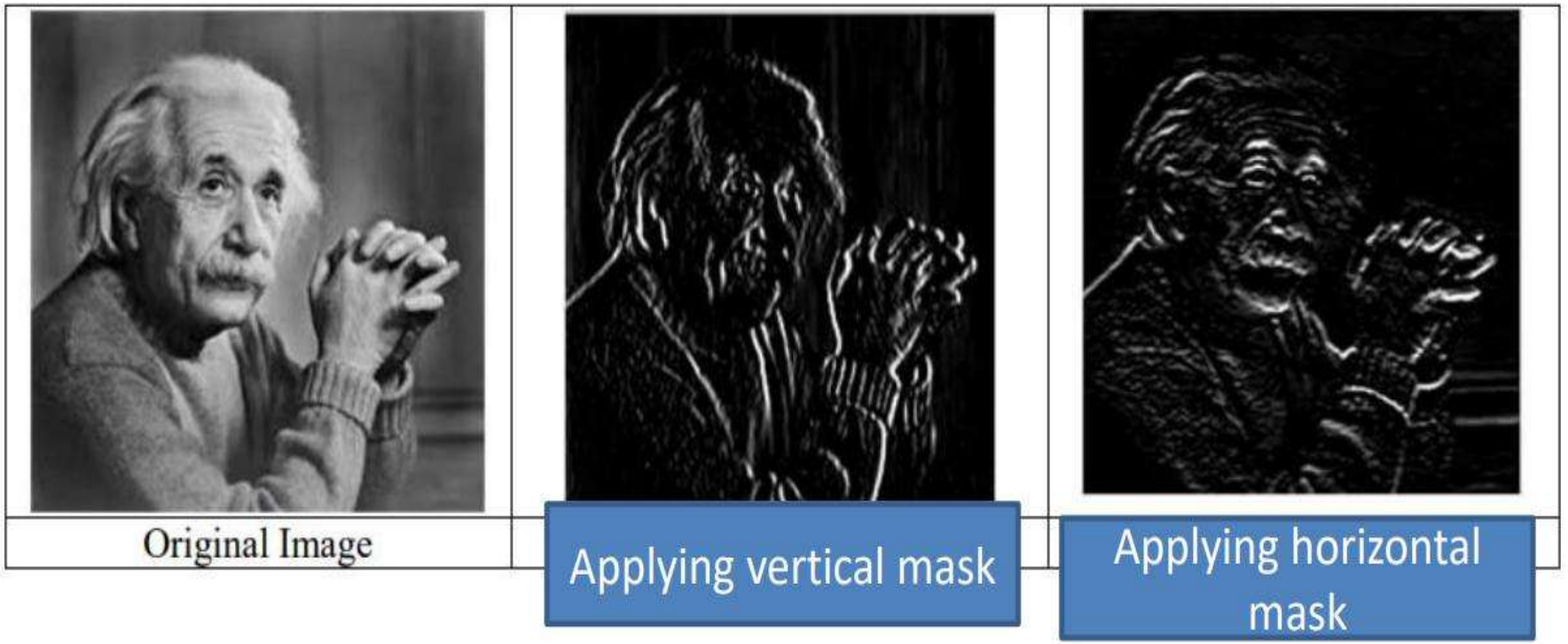
- For vertical edges

-1	0	1
-1	0	1
-1	0	1

- For horizontal edges

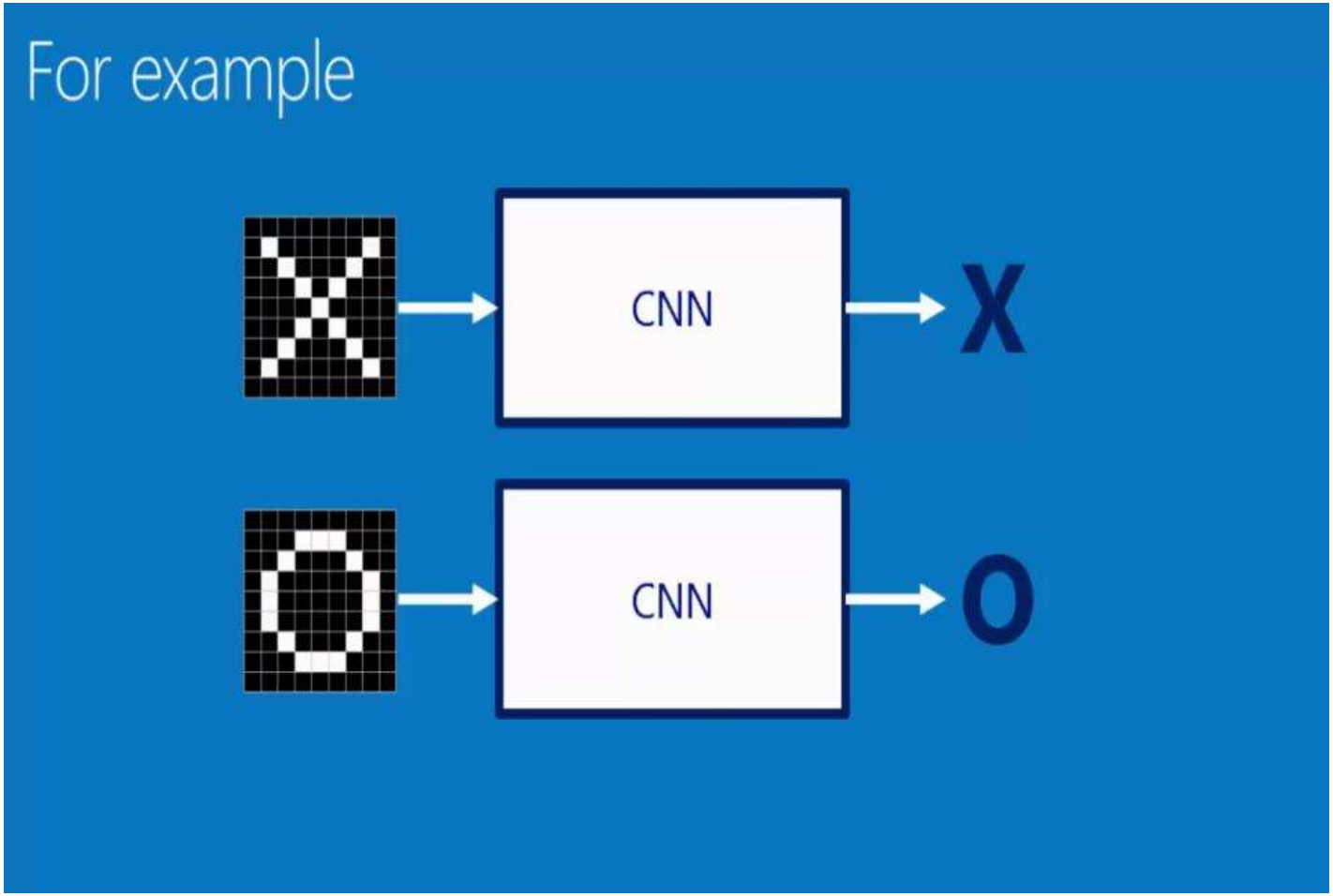
-1	-1	-1
0	0	0
1	1	1

Edges

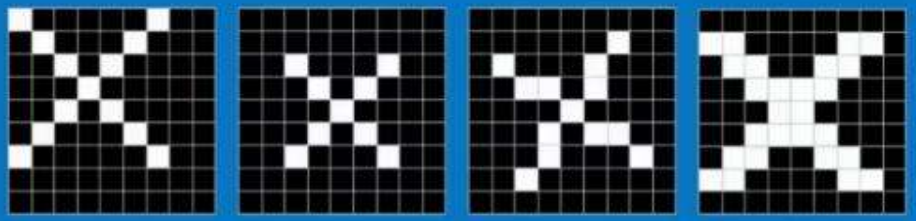

Original Image
Applying vertical mask
Applying horizontal mask

A toy ConvNet: X's and O's

Says whether a picture is of an X or an O


For example
CNN
X
CNN
O

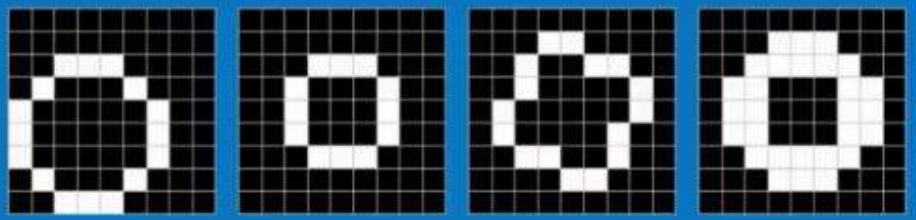
Trickier cases



CNN

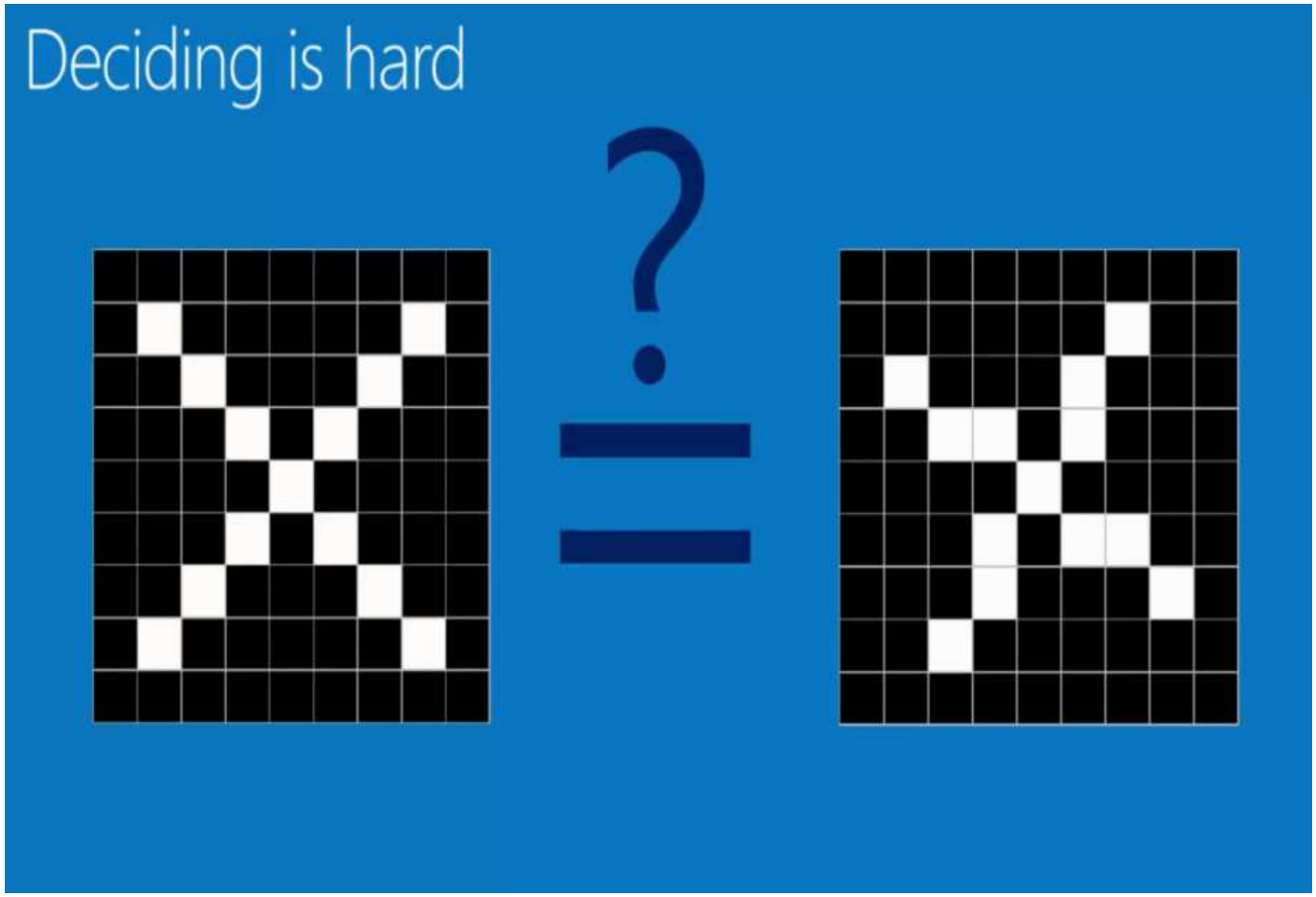
X

translation scaling rotation weight

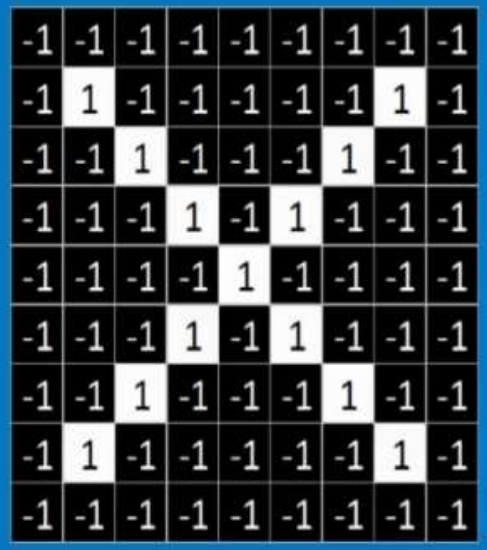


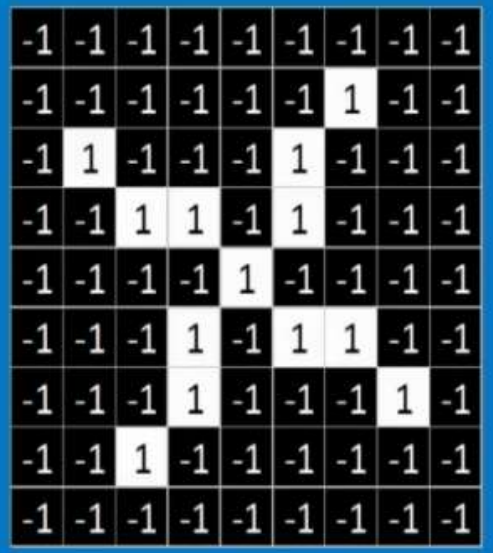
CNN

O

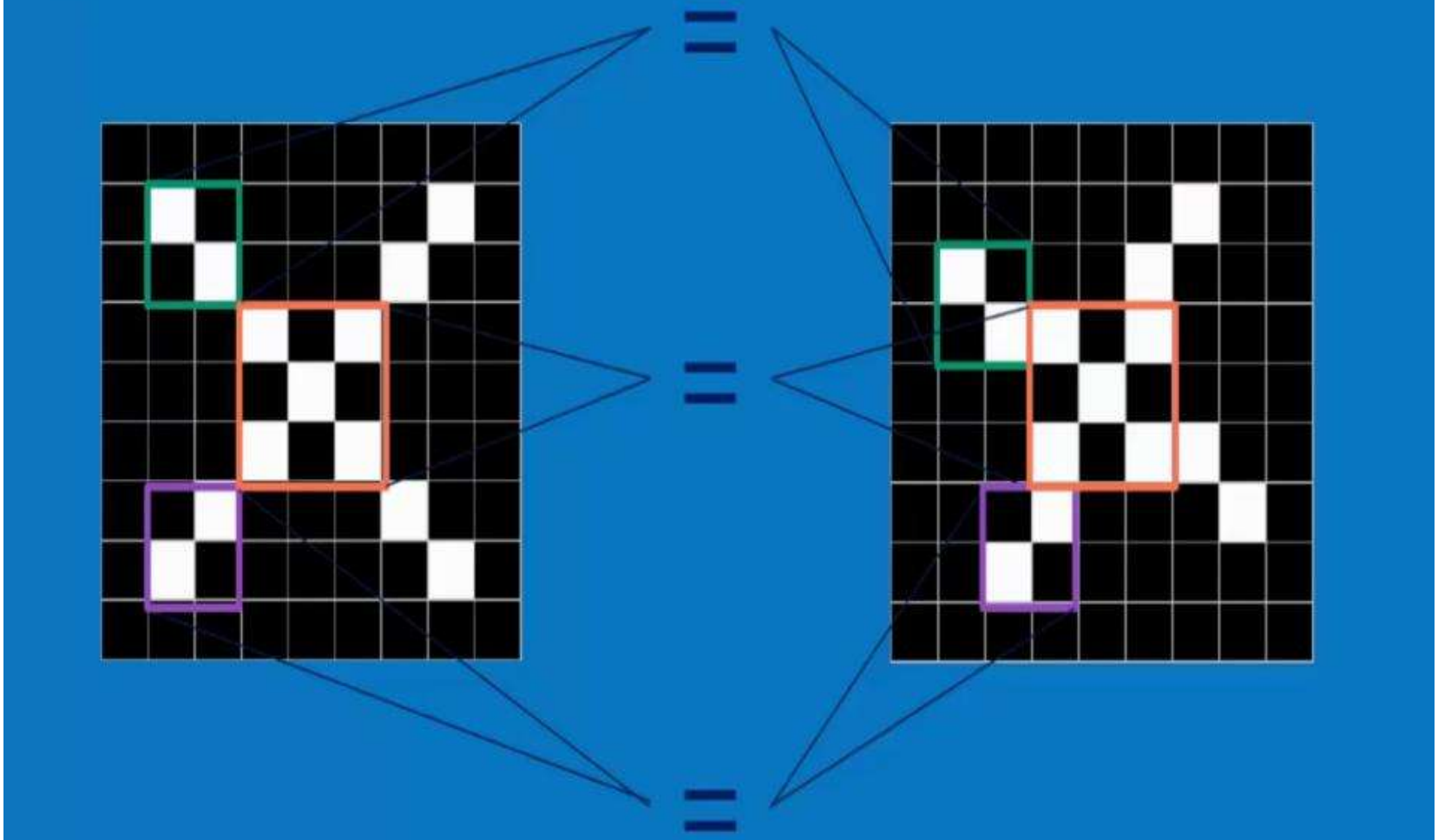

Deciding is hard
?
=

What computers see

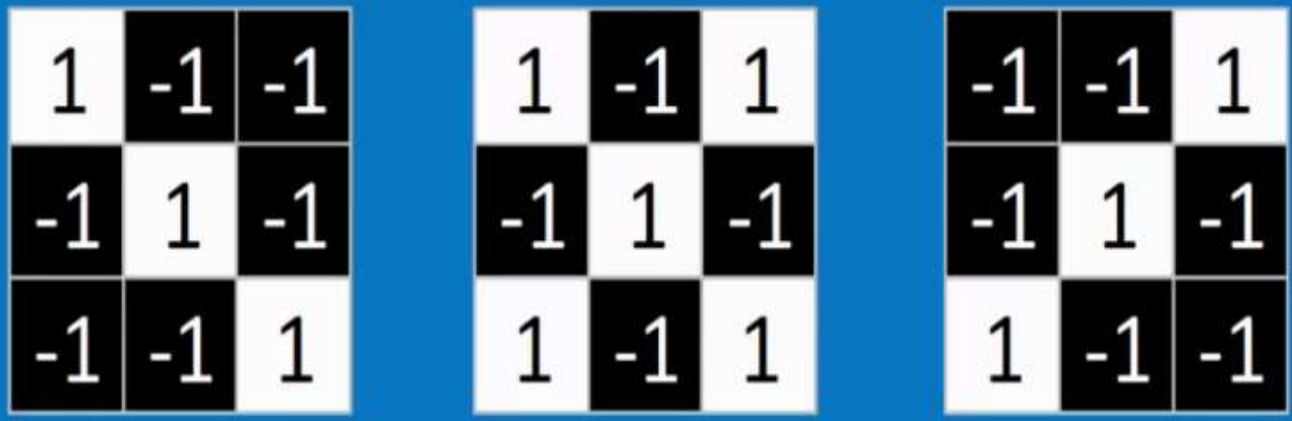


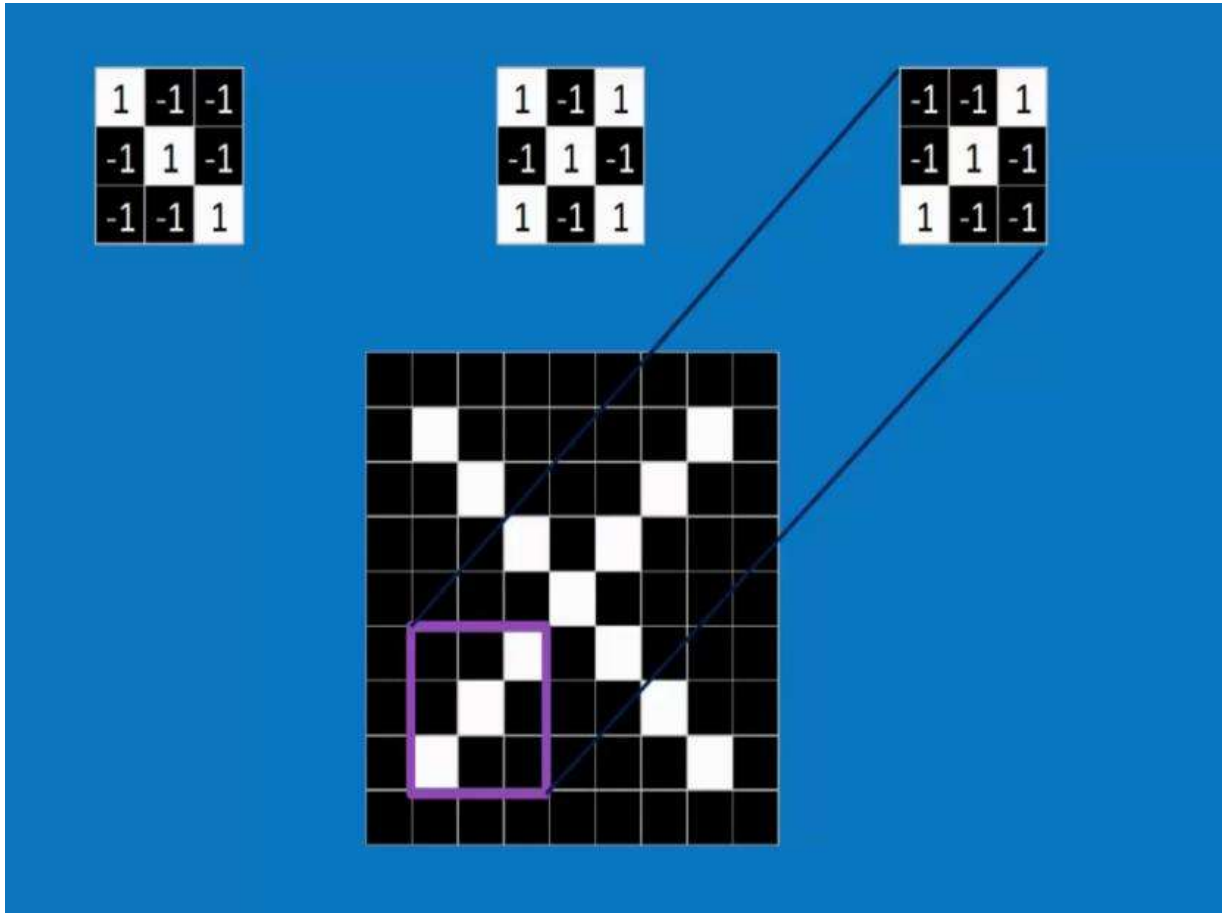


ConvNets match pieces of the image

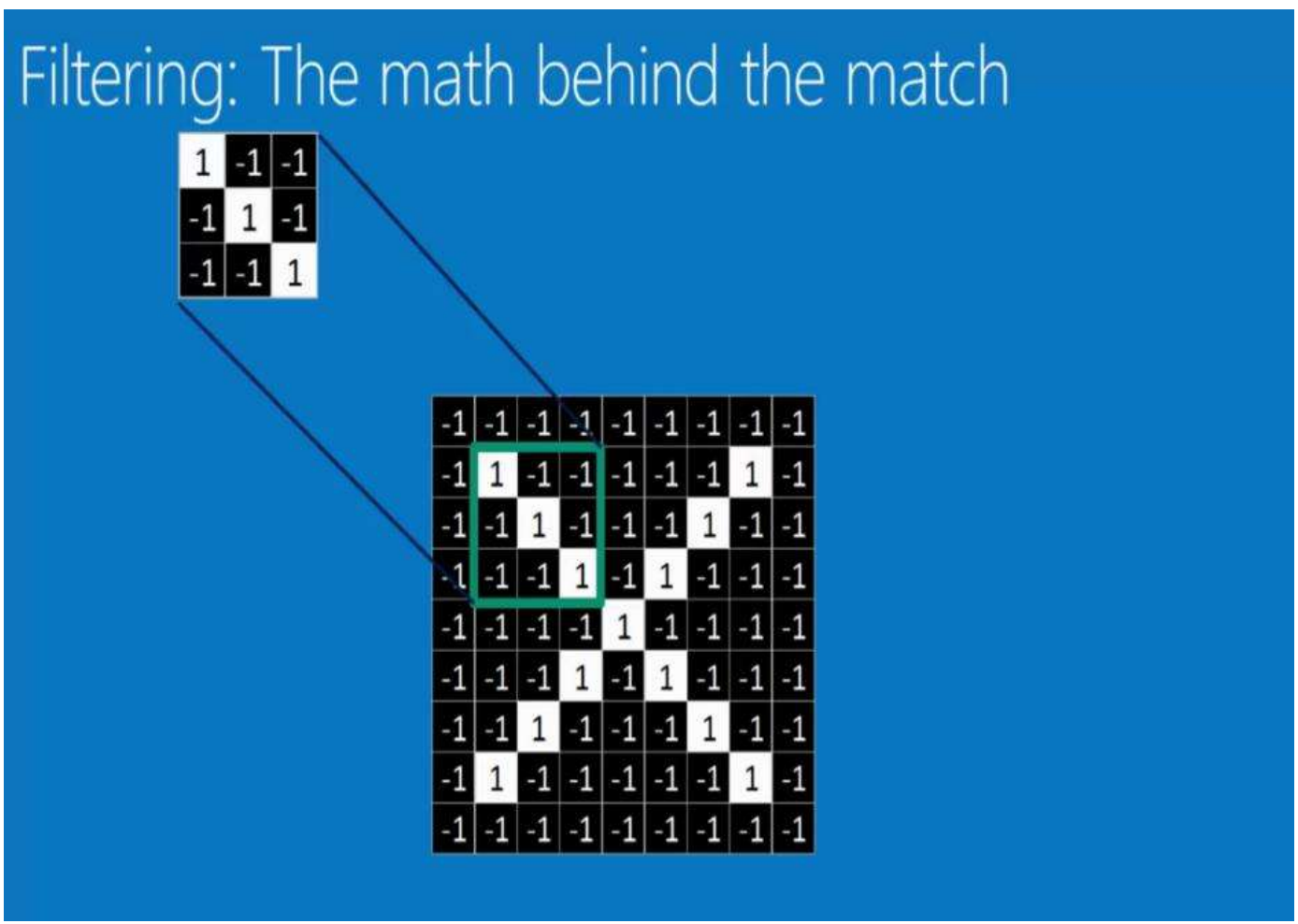


Features match pieces of the image



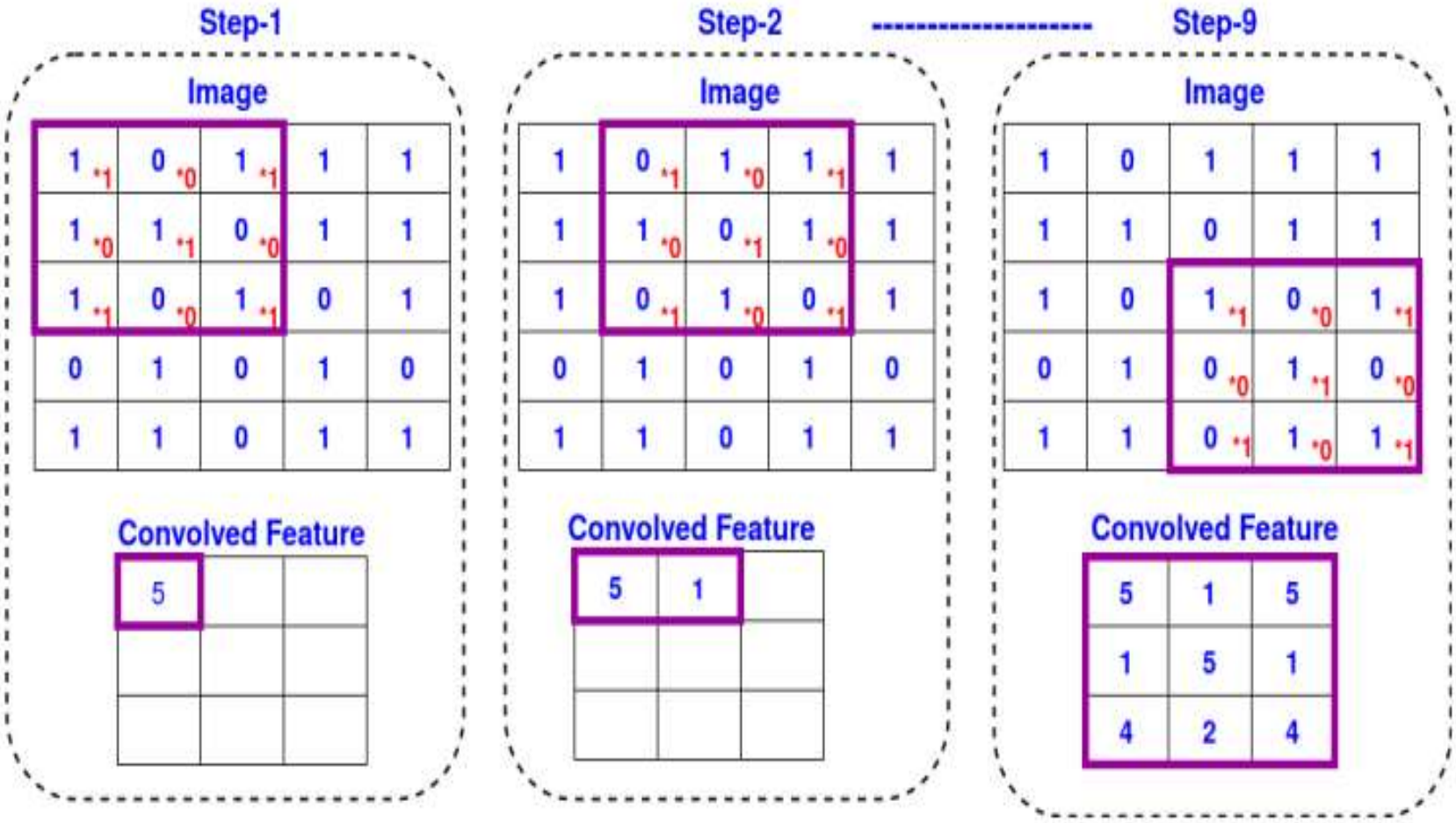

1 -1 -1
-1 1 -1
-1 -1 1
1 -1 1
-1 1 -1
1 -1 1
-1 -1 1
-1 1 -1
1 -1 -1

Filtering: The math behind the match

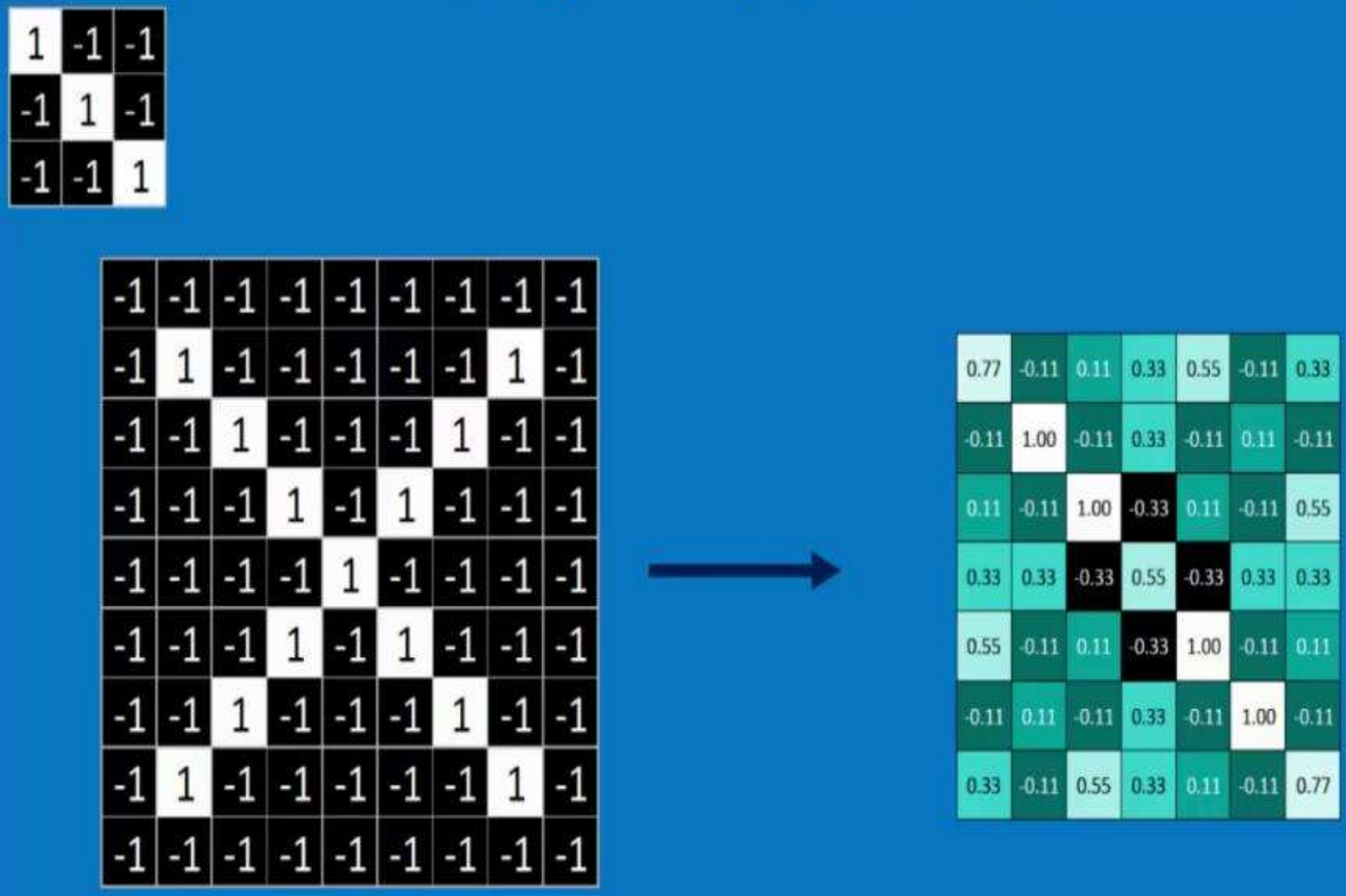


Filtering: The math behind the match

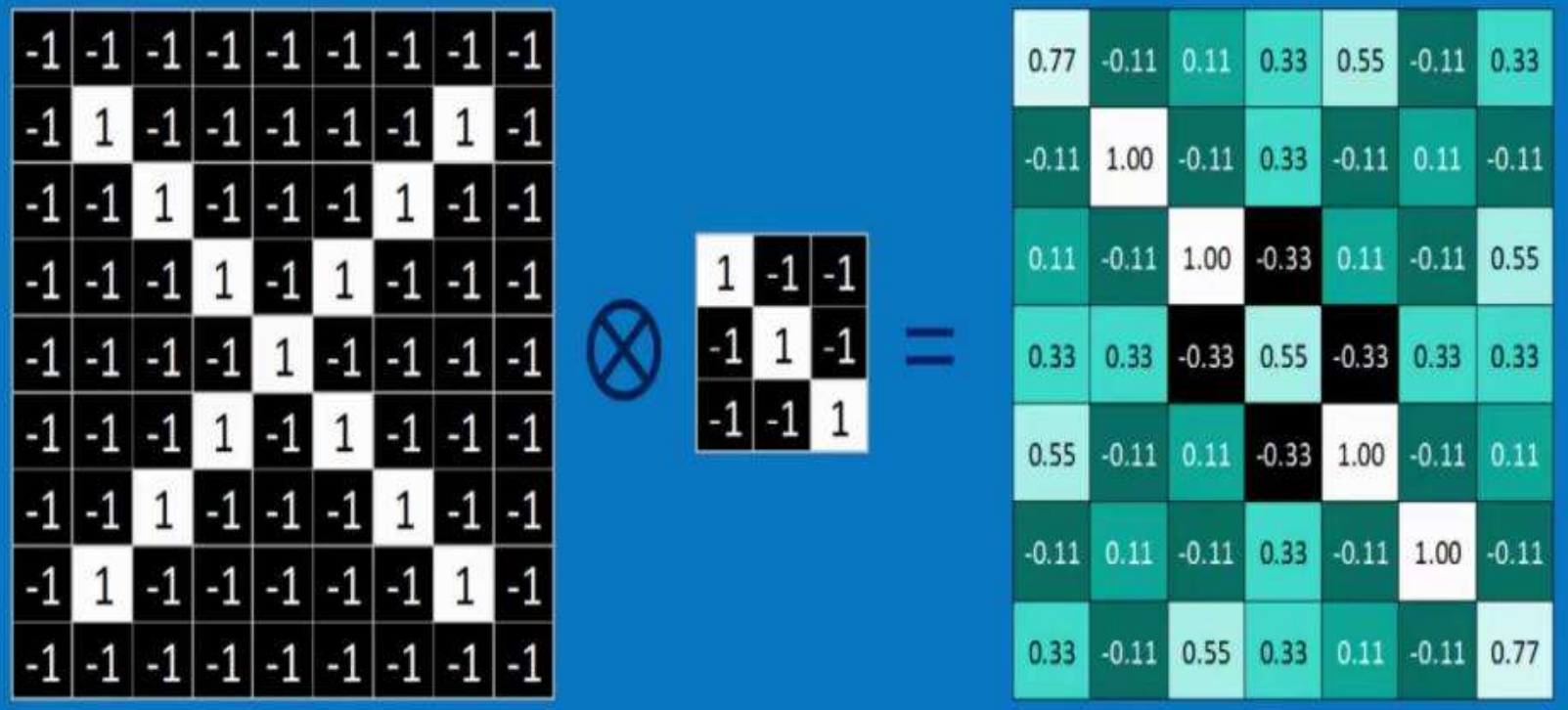
1. Line up the feature and the image patch.
2. Multiply each image pixel by the corresponding feature pixel.
3. Add them up.
4. Divide by the total number of pixels in the feature.

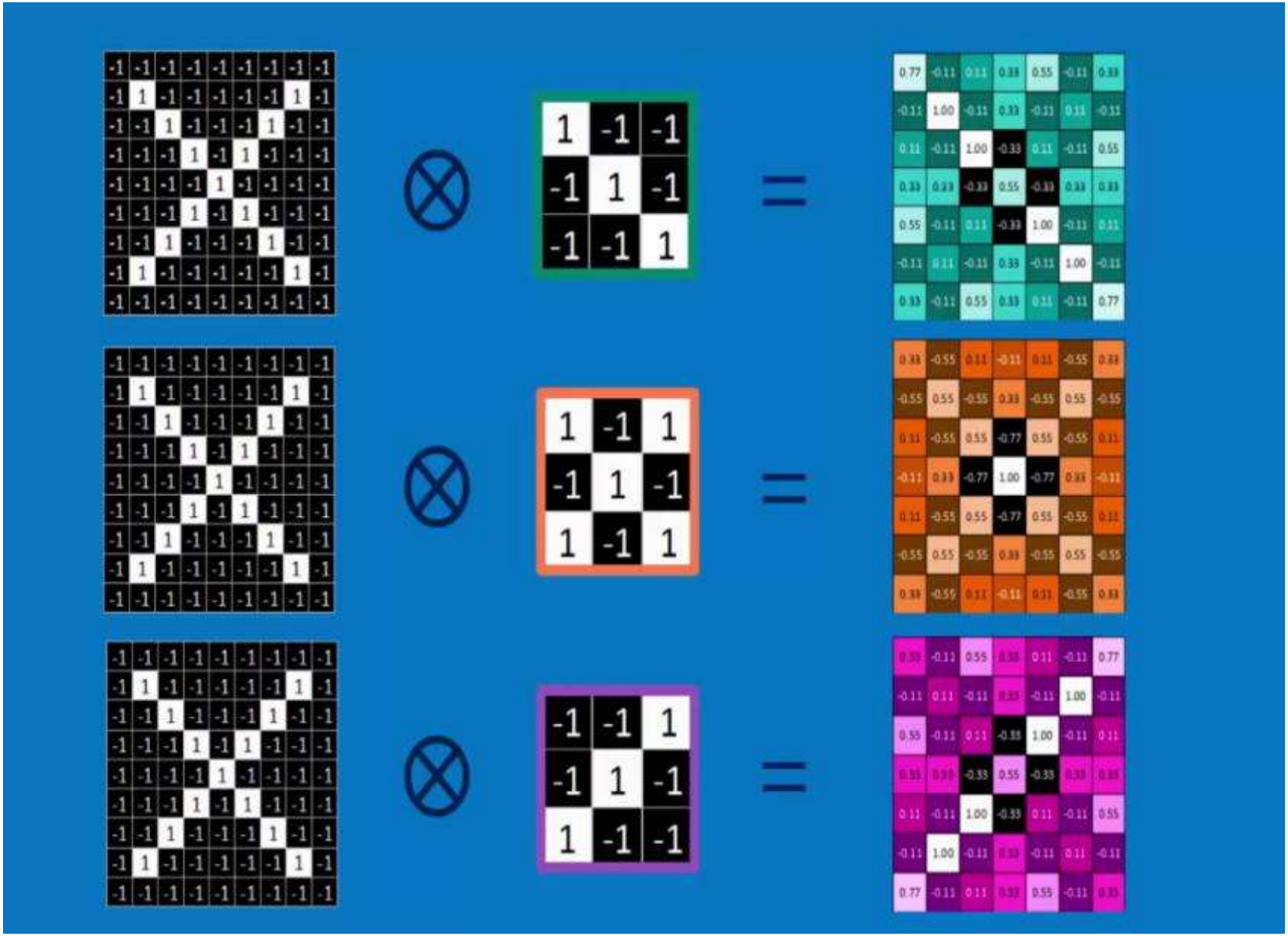

Step-1
Image
1 *1 0 *0 1 *1 1 1
1 *0 1 *1 0 *0 1 1
1 *1 0 *0 1 *1 0 1
0 1 0 1 0
1 1 0 1 1
Convolved Feature
5
Step-2
Image
1 0 *1 1 *0 1 *1 1
1 1 *0 0 *1 1 *0 1
1 0 *1 1 *0 0 *1 1
0 1 0 1 0
1 1 0 1 1
Convolved Feature
5 1
Step-9
Image
1 0 1 1 1
1 1 0 1 1
1 0 1 *1 0 *0 1 *1
0 1 0 *0 1 *1 0 *0
1 1 0 *1 1 *0 1 *1
Convolved Feature
5 1 5
1 5 1
4 2 4

Convolution: Trying every possible match



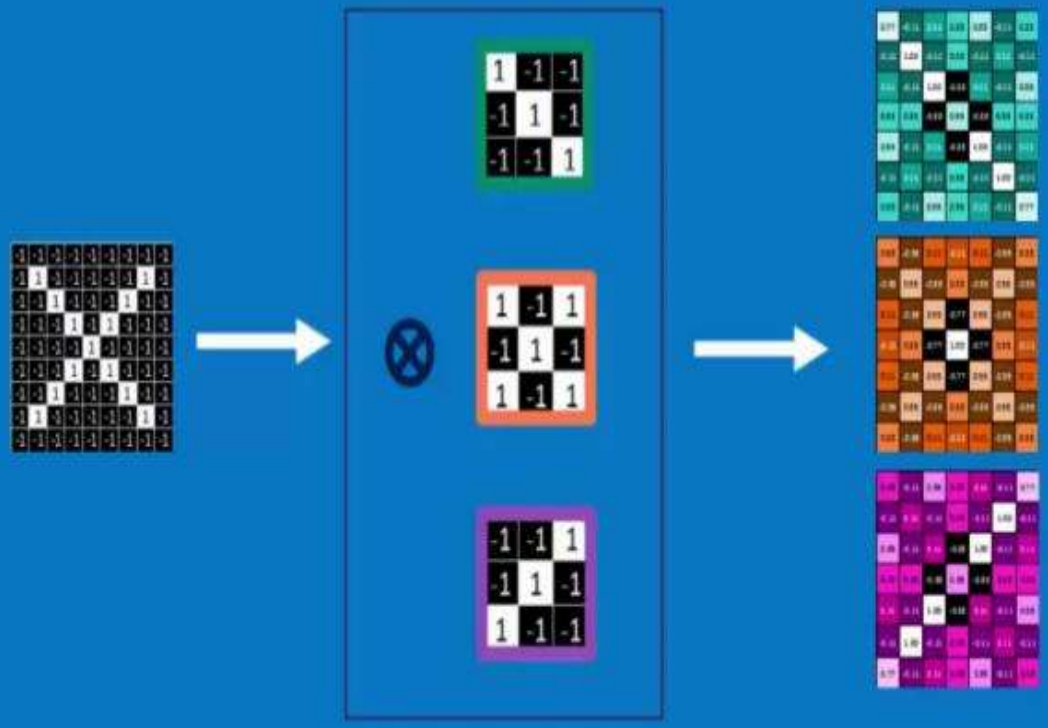
Convolution: Trying every possible match





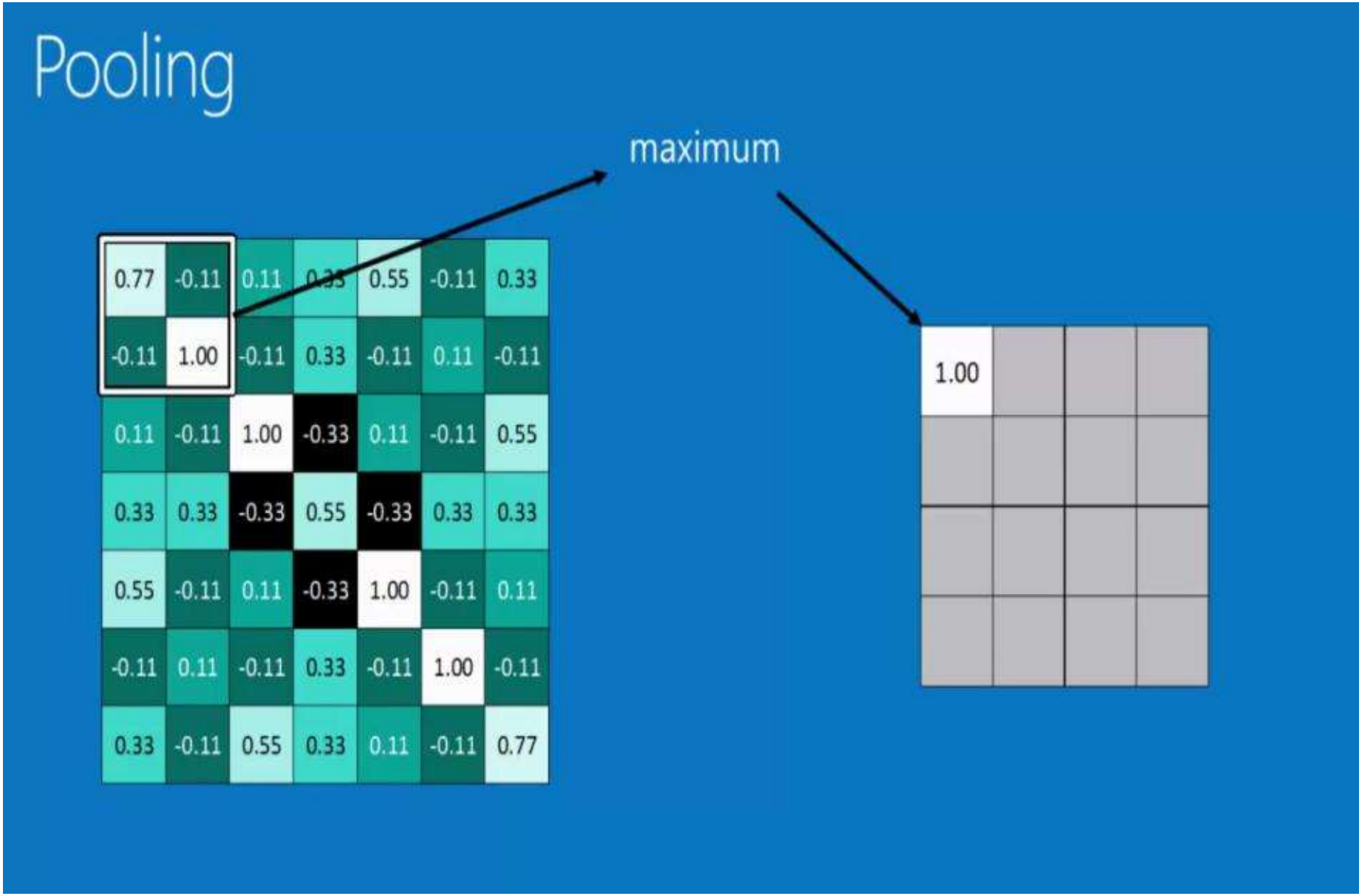
Convolution layer

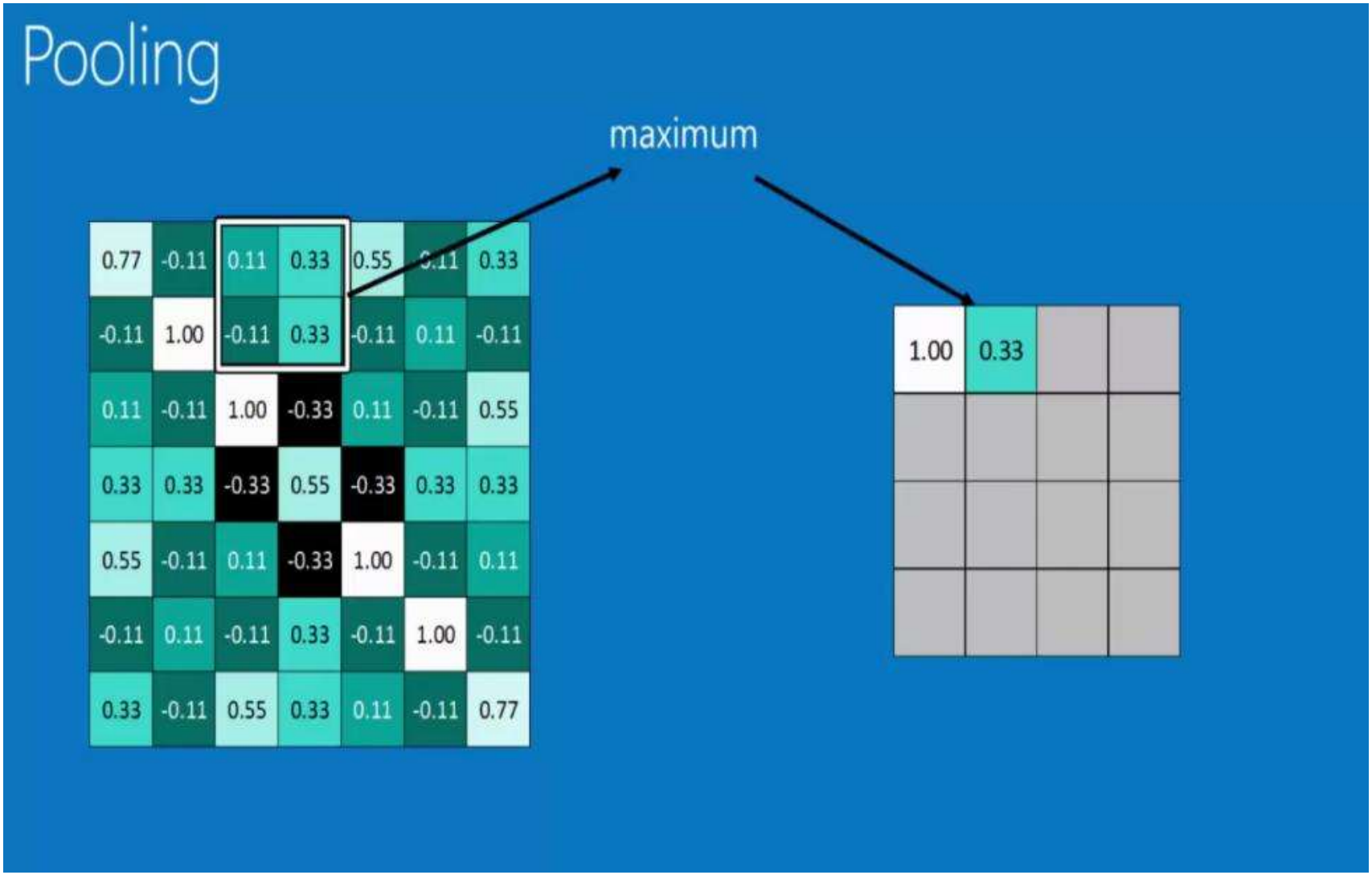
One image becomes a stack of filtered images



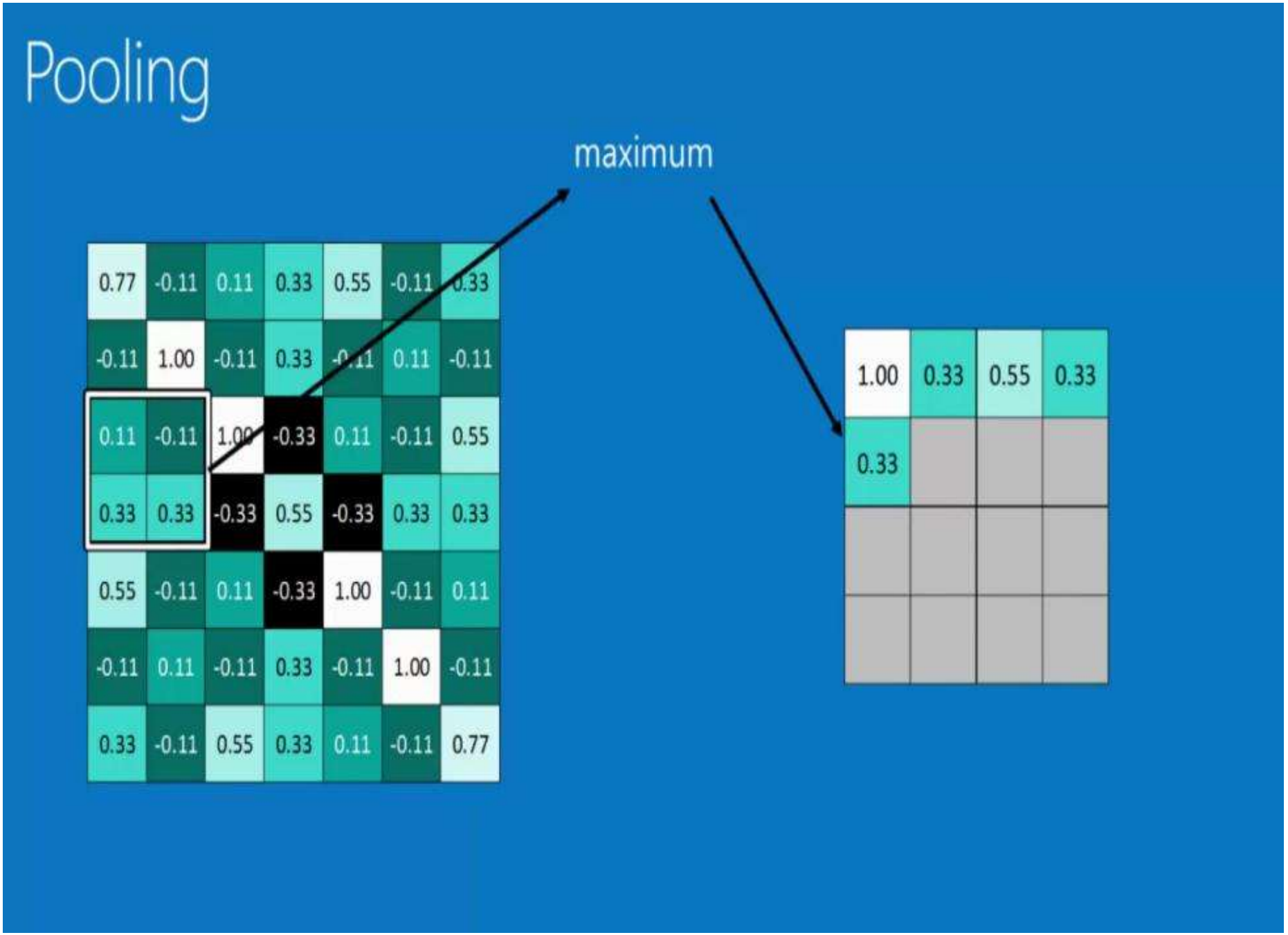
Pooling: Shrinking the image stack

1. Pick a window size (usually 2 or 3).
2. Pick a stride (usually 2).
3. Walk your window across your filtered images.
4. From each window, take the maximum value.

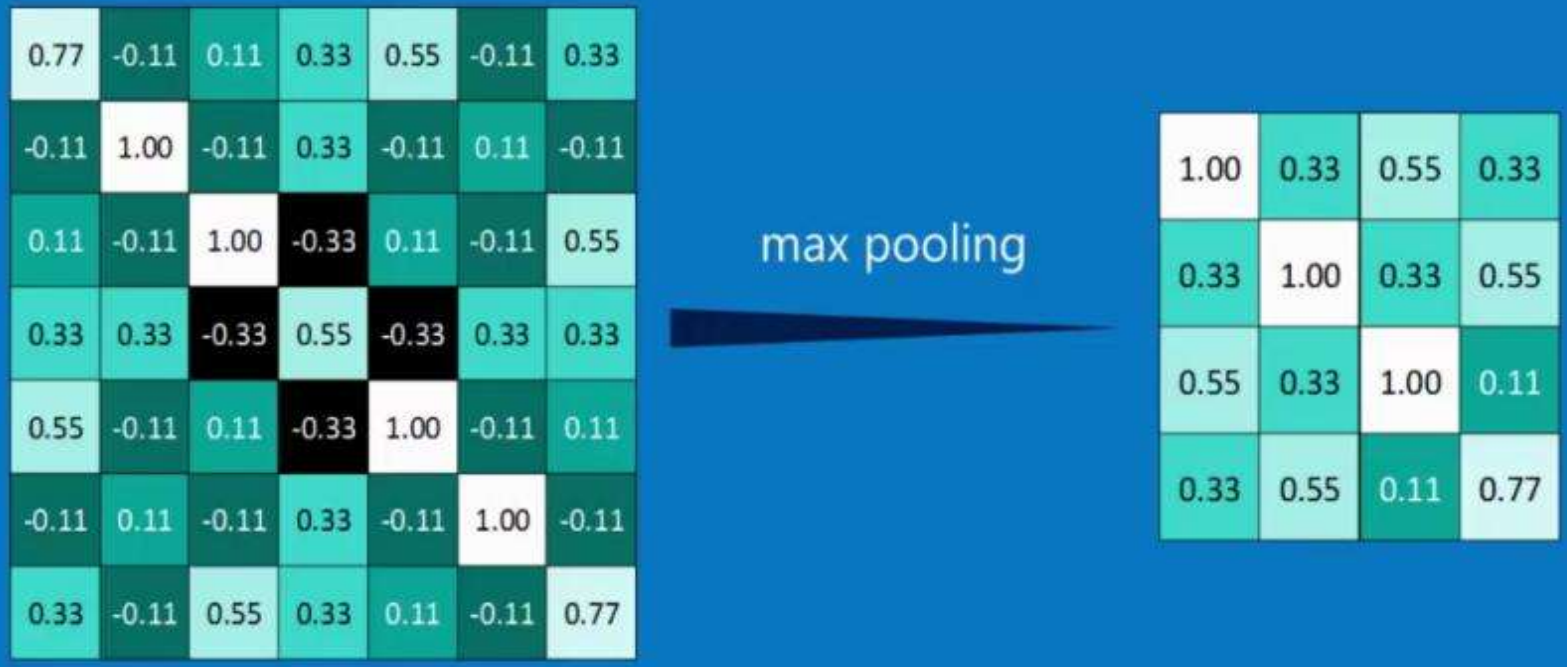

Pooling
maximum
0.77 -0.11 0.11 0.33 0.55 -0.11 0.33
-0.11 1.00 -0.11 0.33 -0.11 0.11 -0.11
0.11 -0.11 1.00 -0.33 0.11 -0.11 0.55
0.33 0.33 -0.33 0.55 -0.33 0.33 0.33
0.55 -0.11 0.11 -0.33 1.00 -0.11 0.11
-0.11 0.11 -0.11 0.33 -0.11 1.00 -0.11
0.33 -0.11 0.55 0.33 0.11 -0.11 0.77
1.00


Pooling
maximum
0.77 -0.11 0.11 0.33 0.55 0.11 0.33
-0.11 1.00 -0.11 0.33 -0.11 0.11 -0.11
0.11 -0.11 1.00 -0.33 0.11 -0.11 0.55
0.33 0.33 -0.33 0.55 -0.33 0.33 0.33
0.55 -0.11 0.11 -0.33 1.00 -0.11 0.11
-0.11 0.11 -0.11 0.33 -0.11 1.00 -0.11
0.33 -0.11 0.55 0.33 0.11 -0.11 0.77
1.00 0.33

Pooling

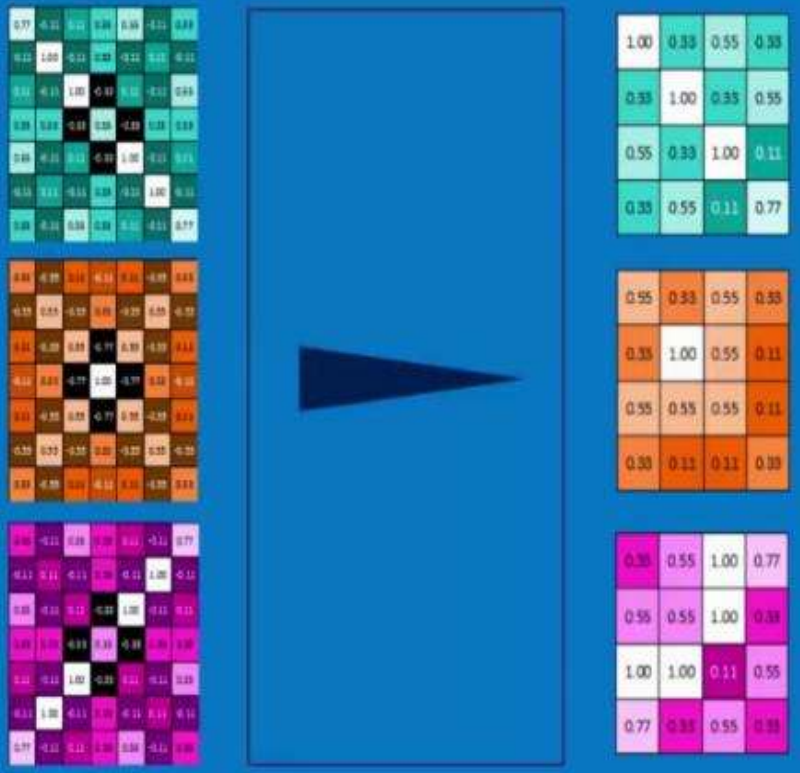

maximum
0.77 -0.11 0.11 0.33 0.55 -0.11 0.33
-0.11 1.00 -0.11 0.33 -0.11 0.11 -0.11
0.11 -0.11 1.00 -0.33 0.11 -0.11 0.55
0.33 0.33 -0.33 0.55 -0.33 0.33 0.33
0.55 -0.11 0.11 -0.33 1.00 -0.11 0.11
-0.11 0.11 -0.11 0.33 -0.11 1.00 -0.11
0.33 -0.11 0.55 0.33 0.11 -0.11 0.77
1.00 0.33 0.55 0.33
0.33

Pooling

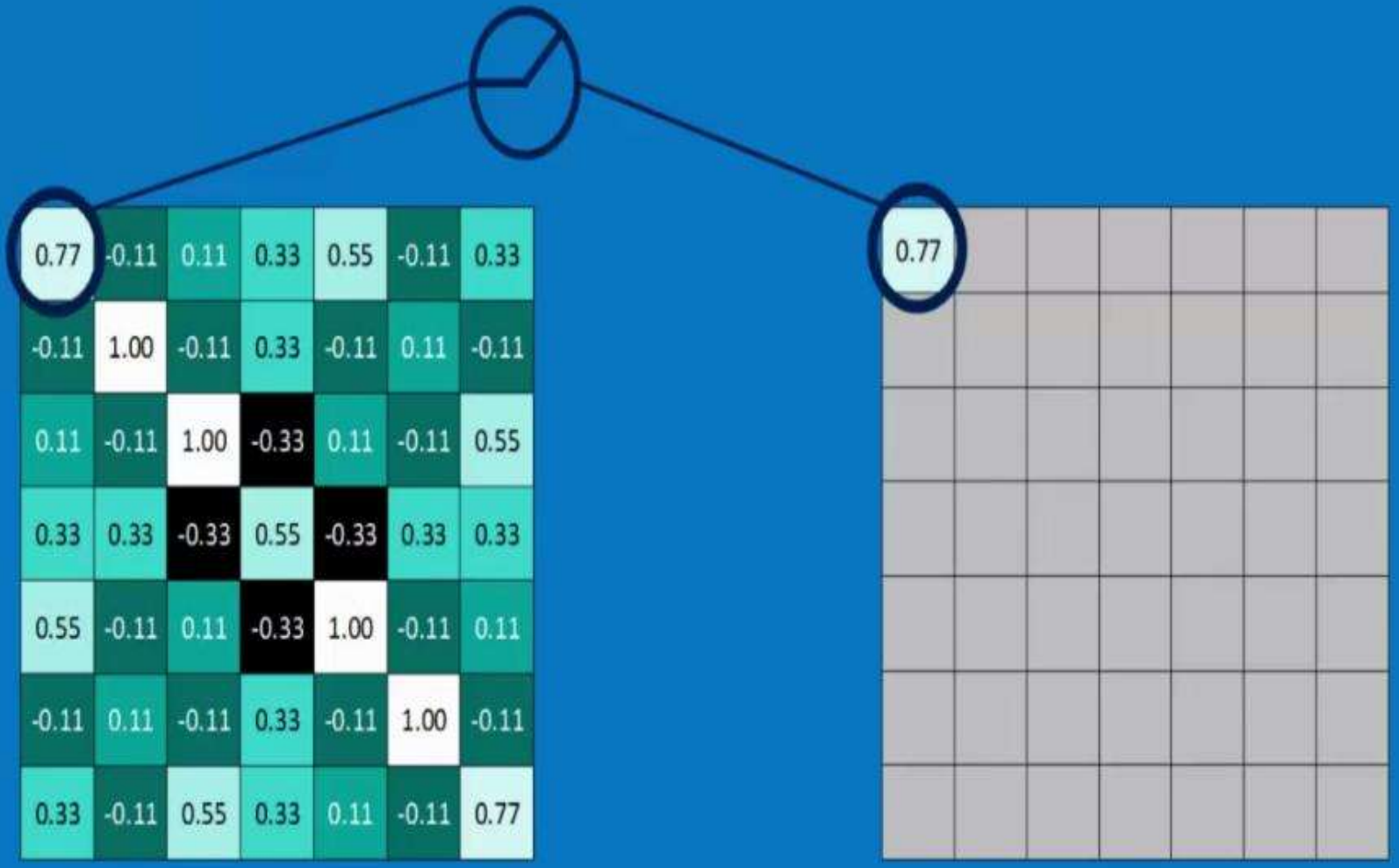

0.77 -0.11 0.11 0.33 0.55 -0.11 0.33
-0.11 1.00 -0.11 0.33 -0.11 0.11 -0.11
0.11 -0.11 1.00 -0.33 0.11 -0.11 0.55
0.33 0.33 -0.33 0.55 -0.33 0.33 0.33
0.55 -0.11 0.11 -0.33 1.00 -0.11 0.11
-0.11 0.11 -0.11 0.33 -0.11 1.00 -0.11
0.33 -0.11 0.55 0.33 0.11 -0.11 0.77
max pooling
1.00 0.33 0.55 0.33
0.33 1.00 0.33 0.55
0.55 0.33 1.00 0.11
0.33 0.55 0.11 0.77

Pooling layer

A stack of images becomes a stack of smaller images.



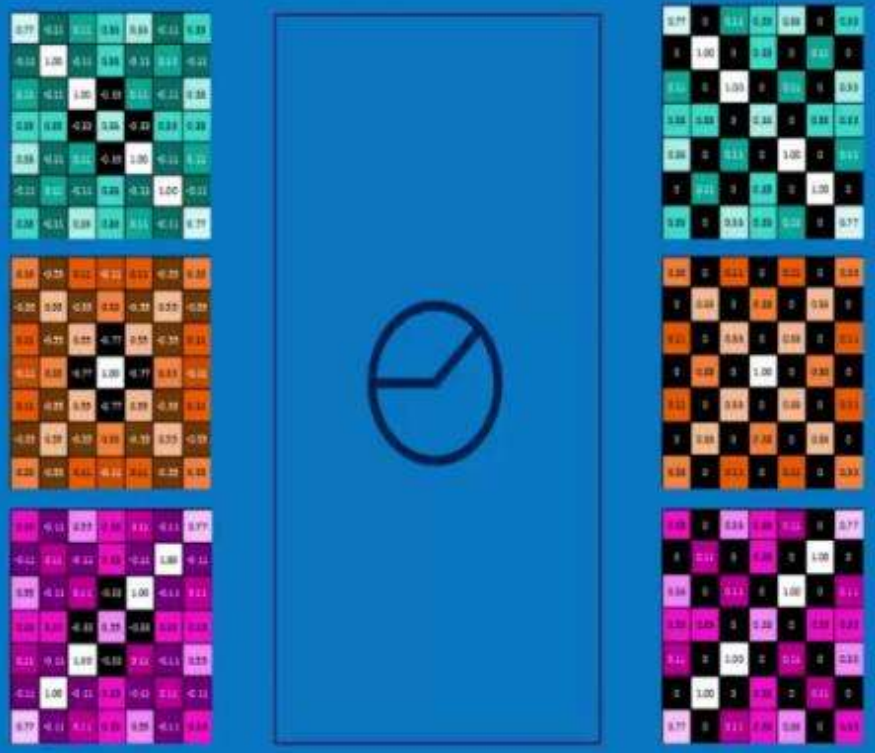
Rectified Linear Units (ReLUs)



Rectified Linear Units (ReLUs)

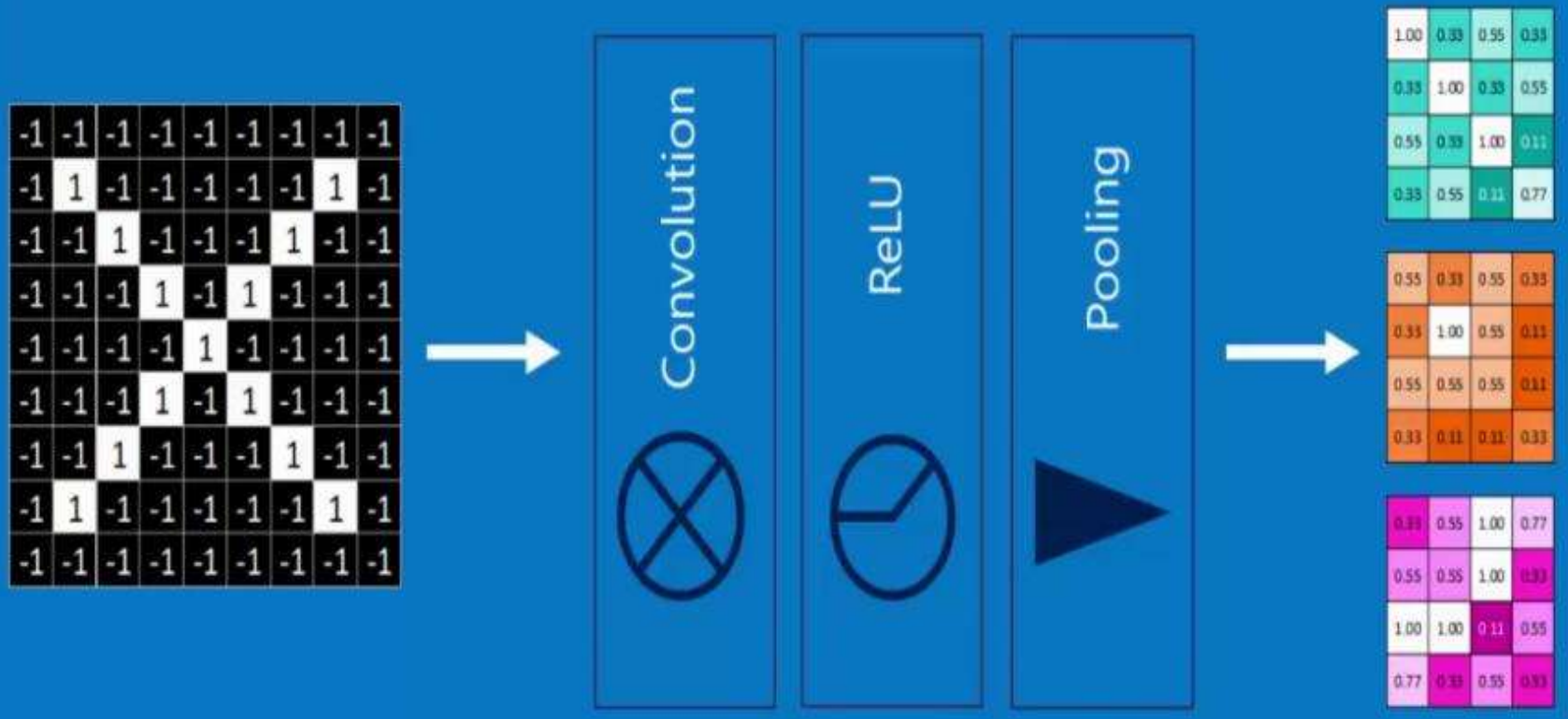
ReLU layer

A stack of images becomes a stack of images with no negative values.



Layers get stacked

The output of one becomes the input of the next.

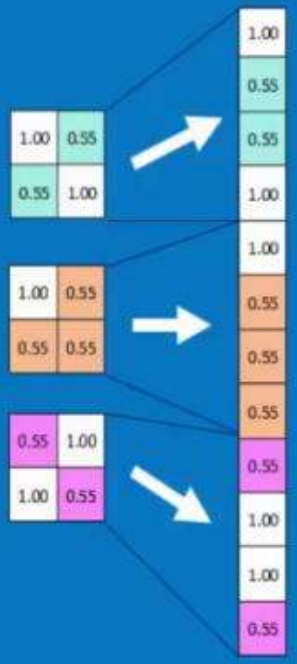


Deep stacking

Layers can be repeated several (or many) times.

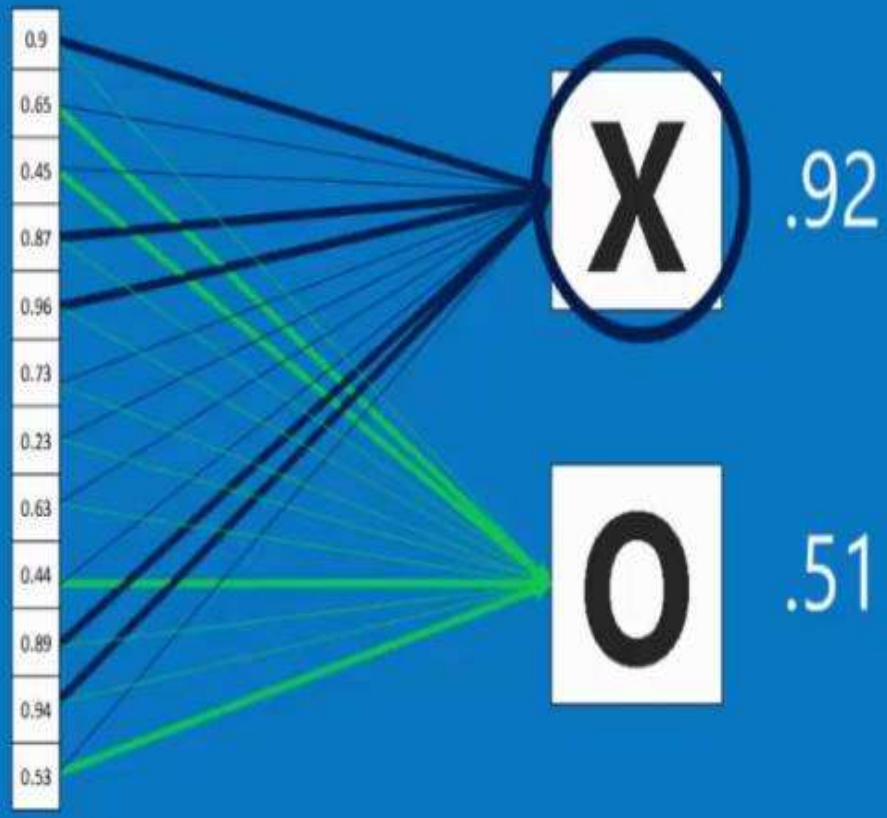
Fully connected layer

Every value gets a vote



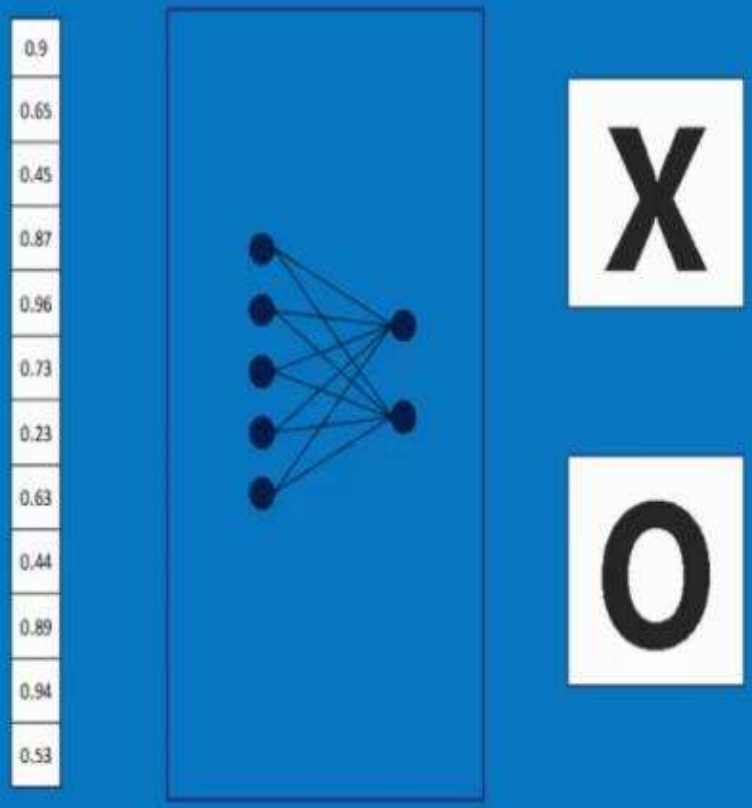
Fully connected layer

Future values vote on X or O



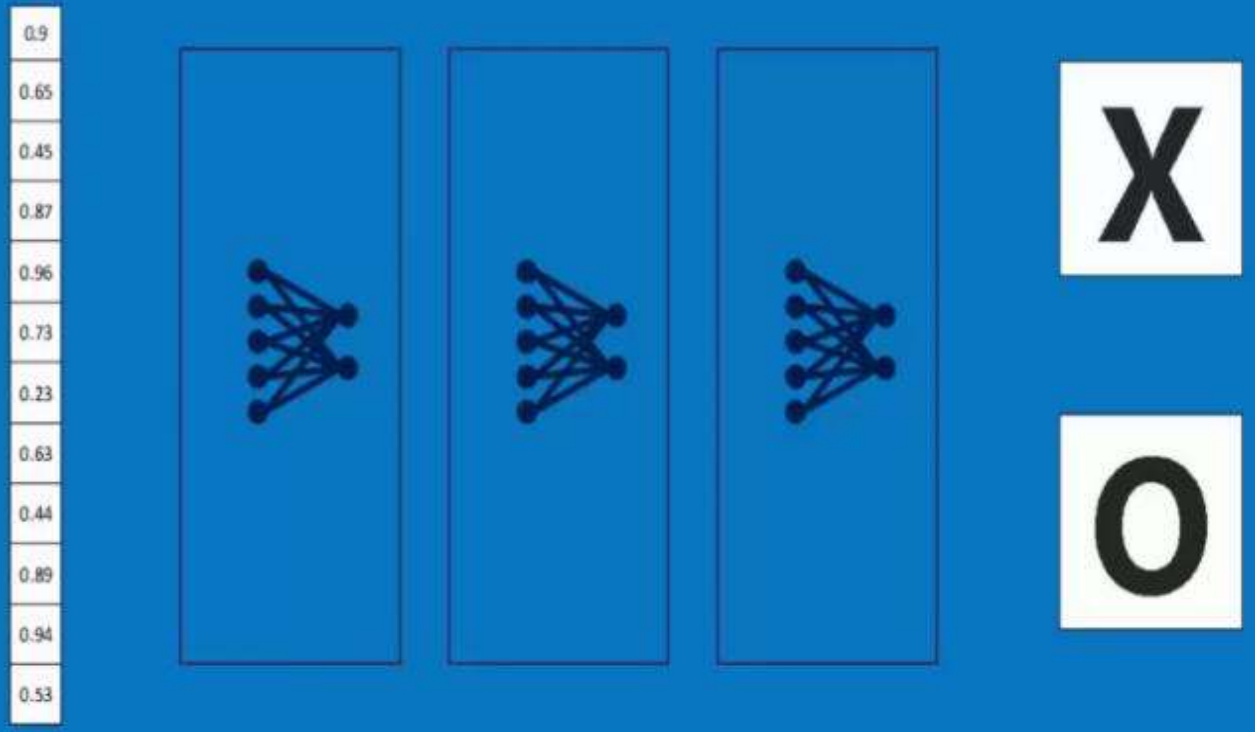
Fully connected layer

A list of feature values becomes a list of votes.



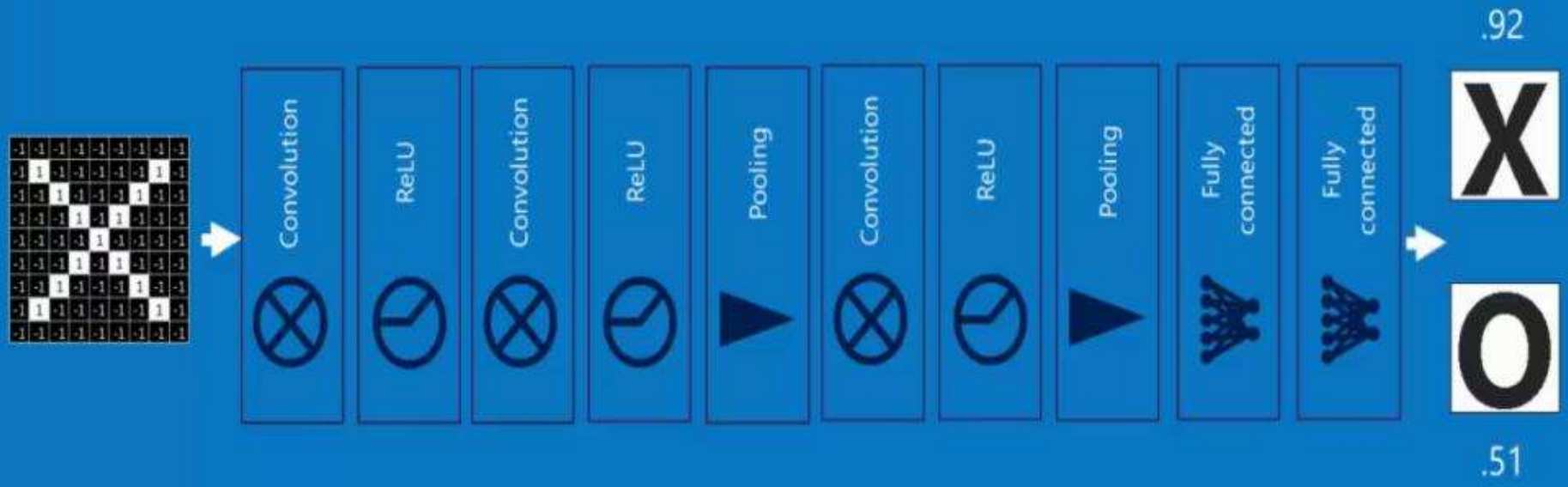
Fully connected layer

These can also be stacked.



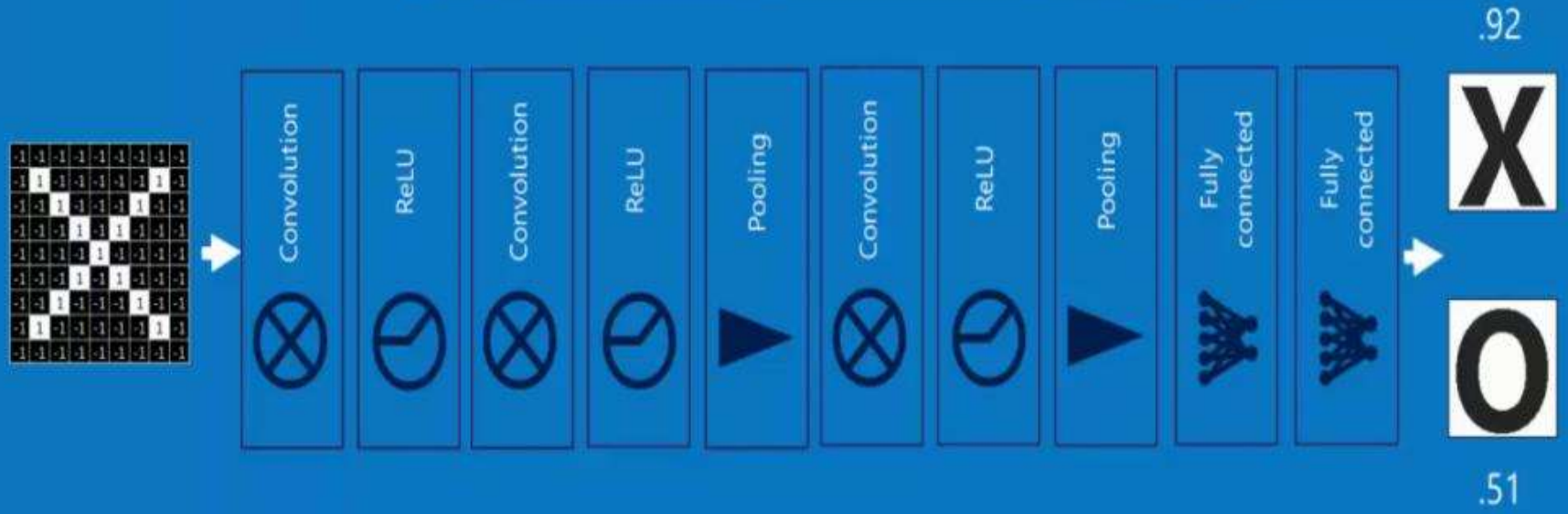
Putting it all together

A set of pixels becomes a set of votes.



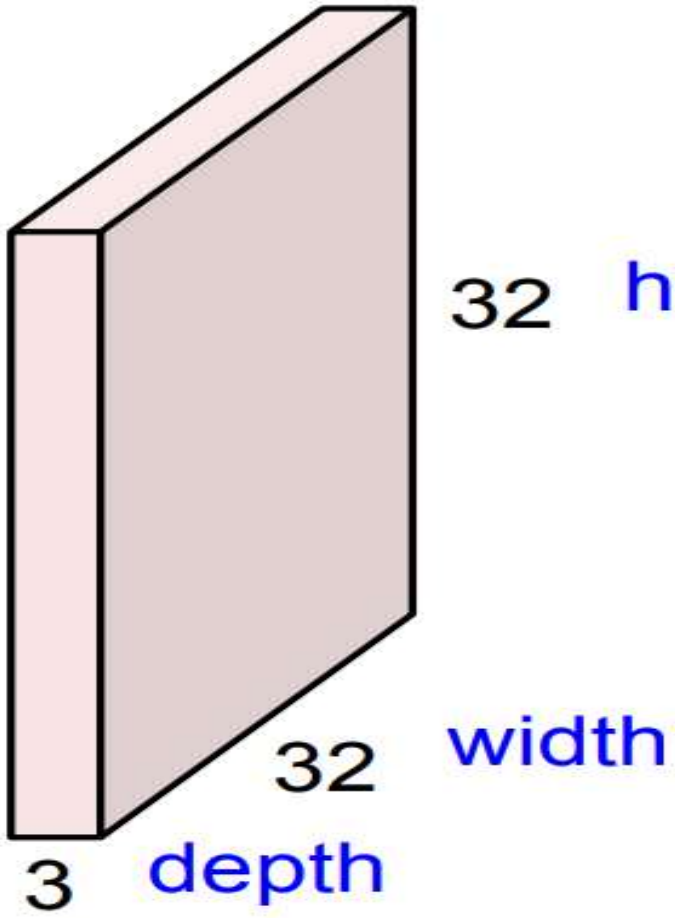
Backprop

	Right answer	Actual answer	Error
X	1	0.92	0.08
O	0	0.51	0.49
		Total	0.57



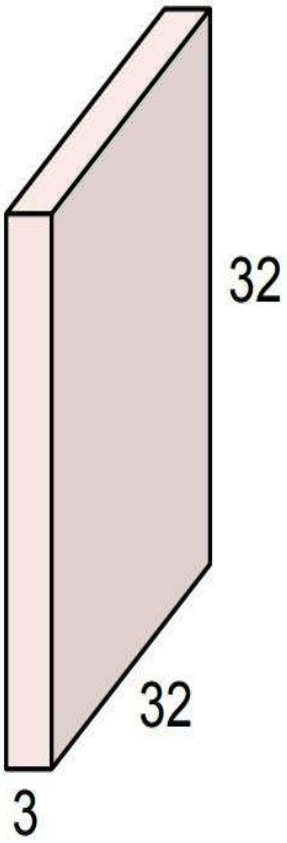
Convolution Layer

32x32x3 image

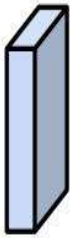


Convolution Layer

32x32x3 image

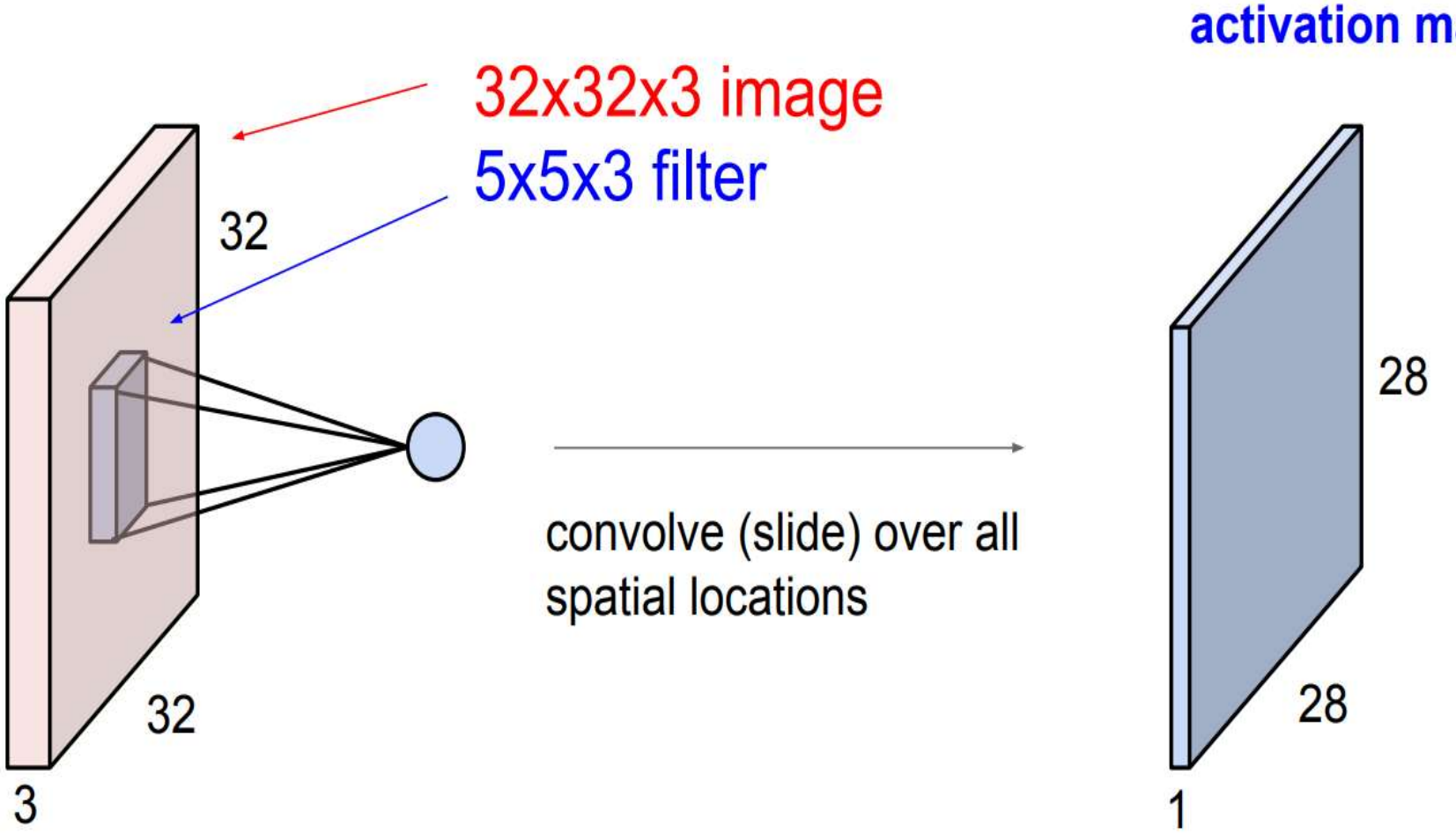


5x5x3 filter



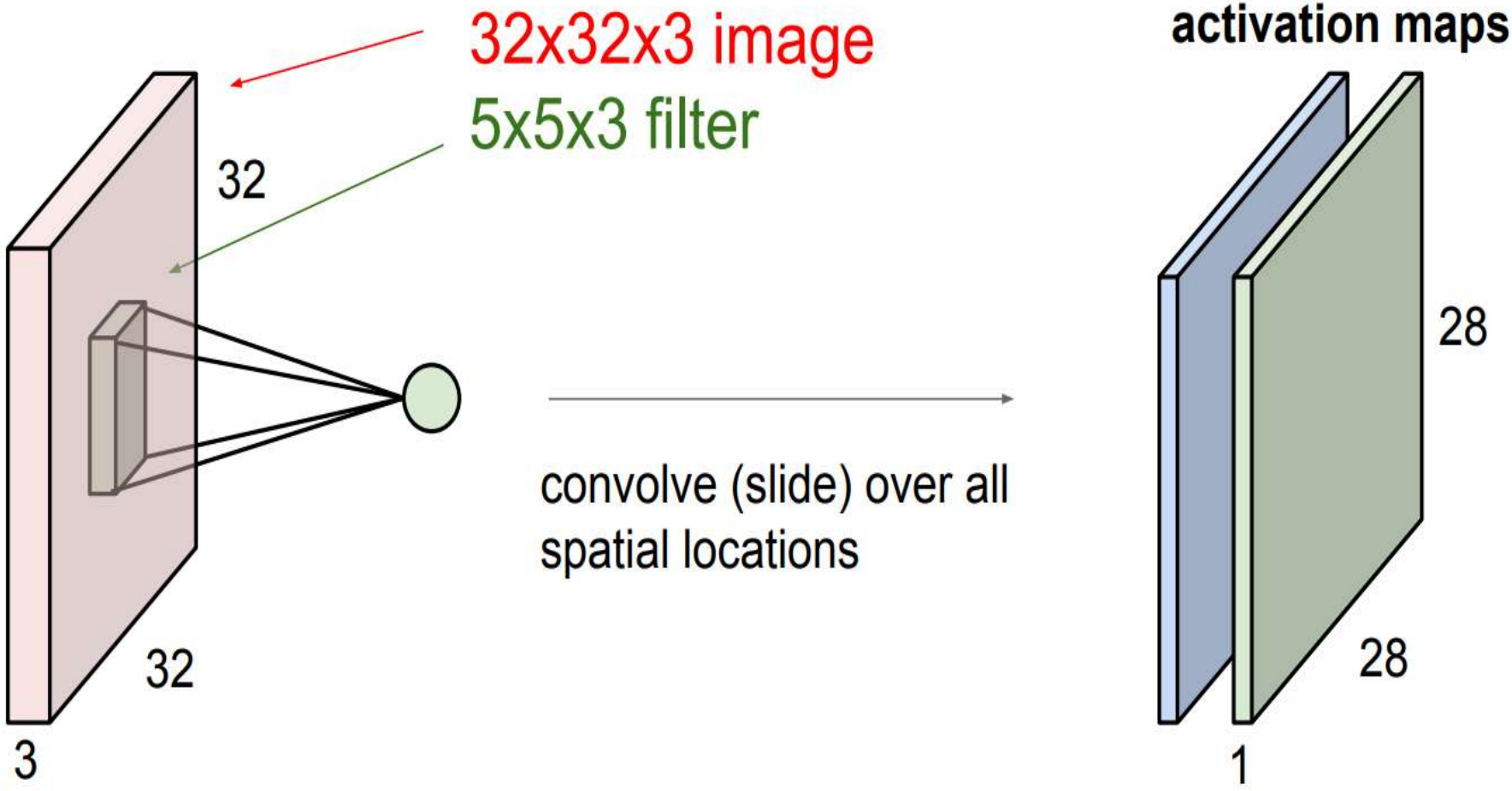
Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

Convolution Layer

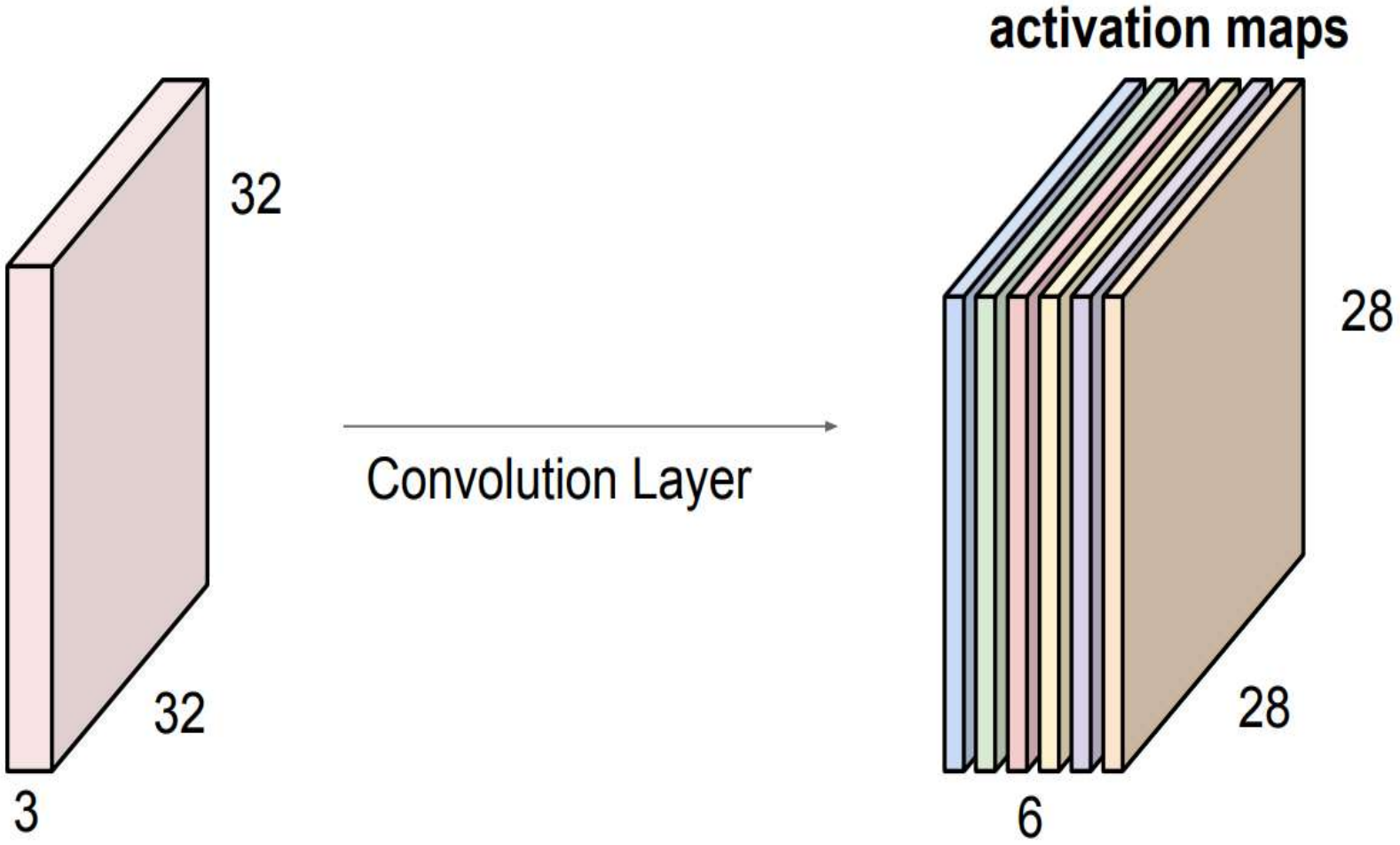


Convolution Layer

consider a second, green filter

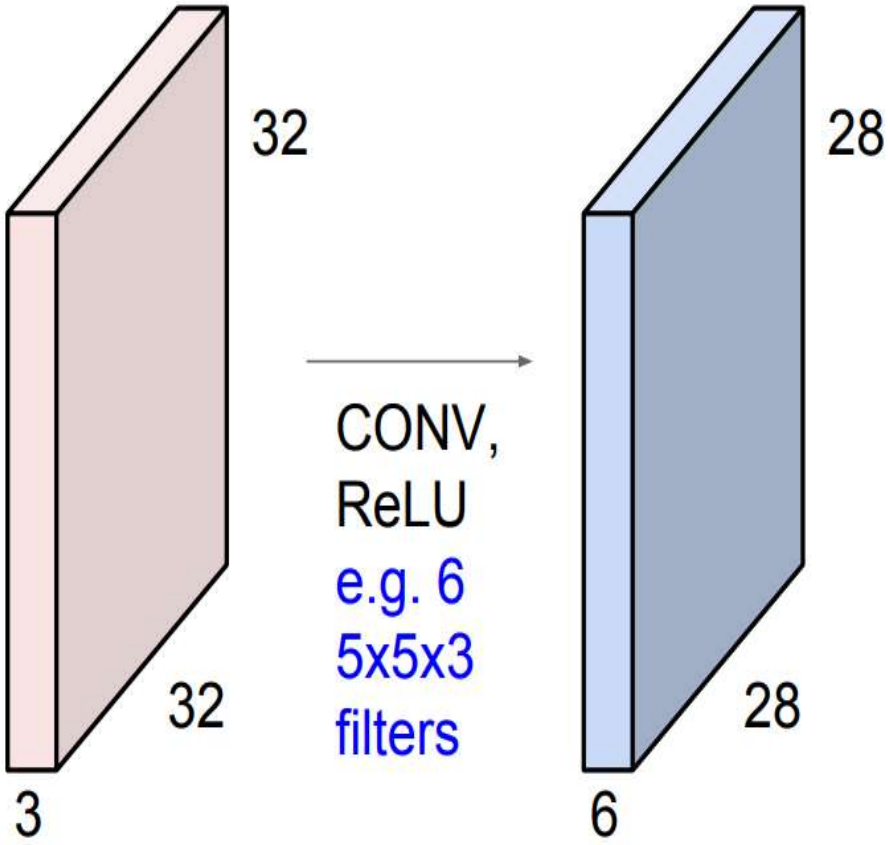


For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:

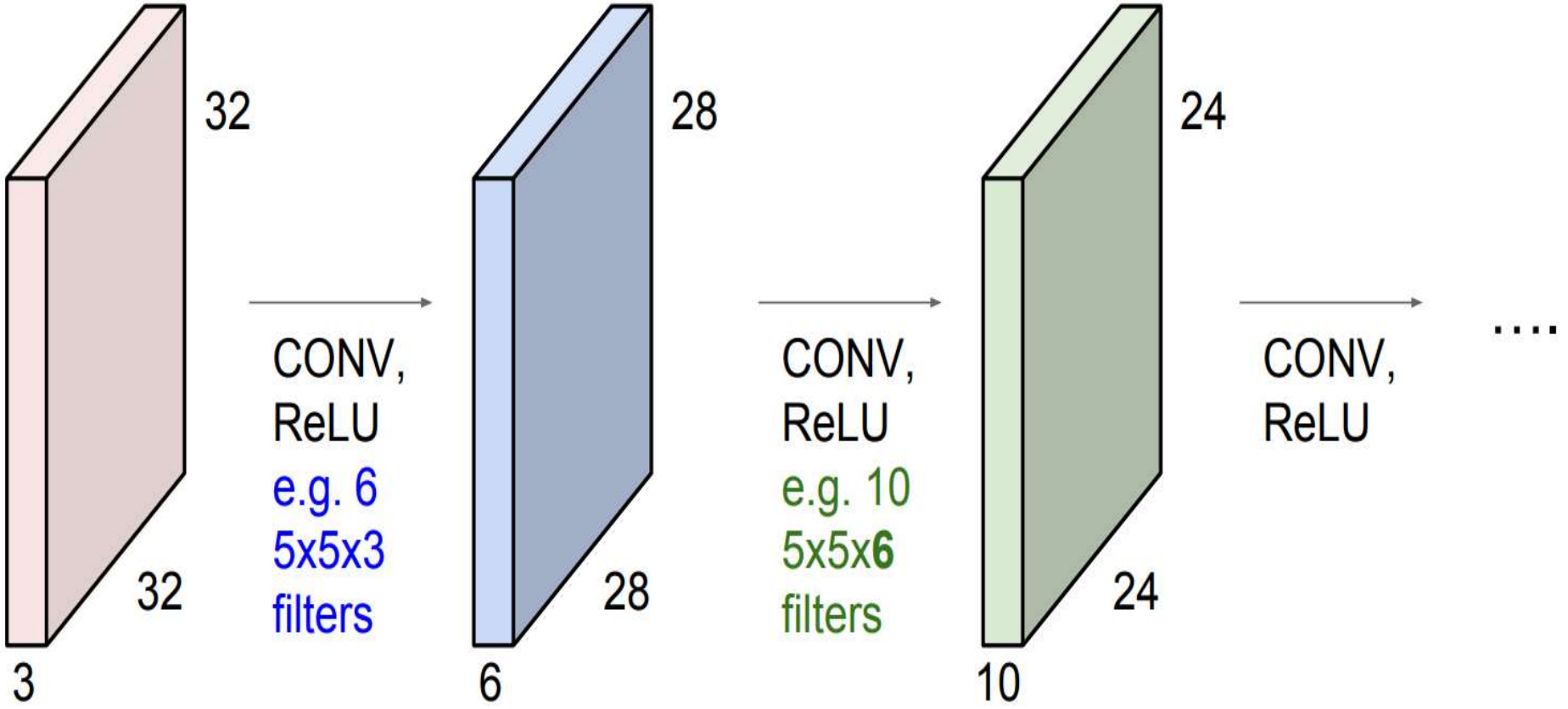


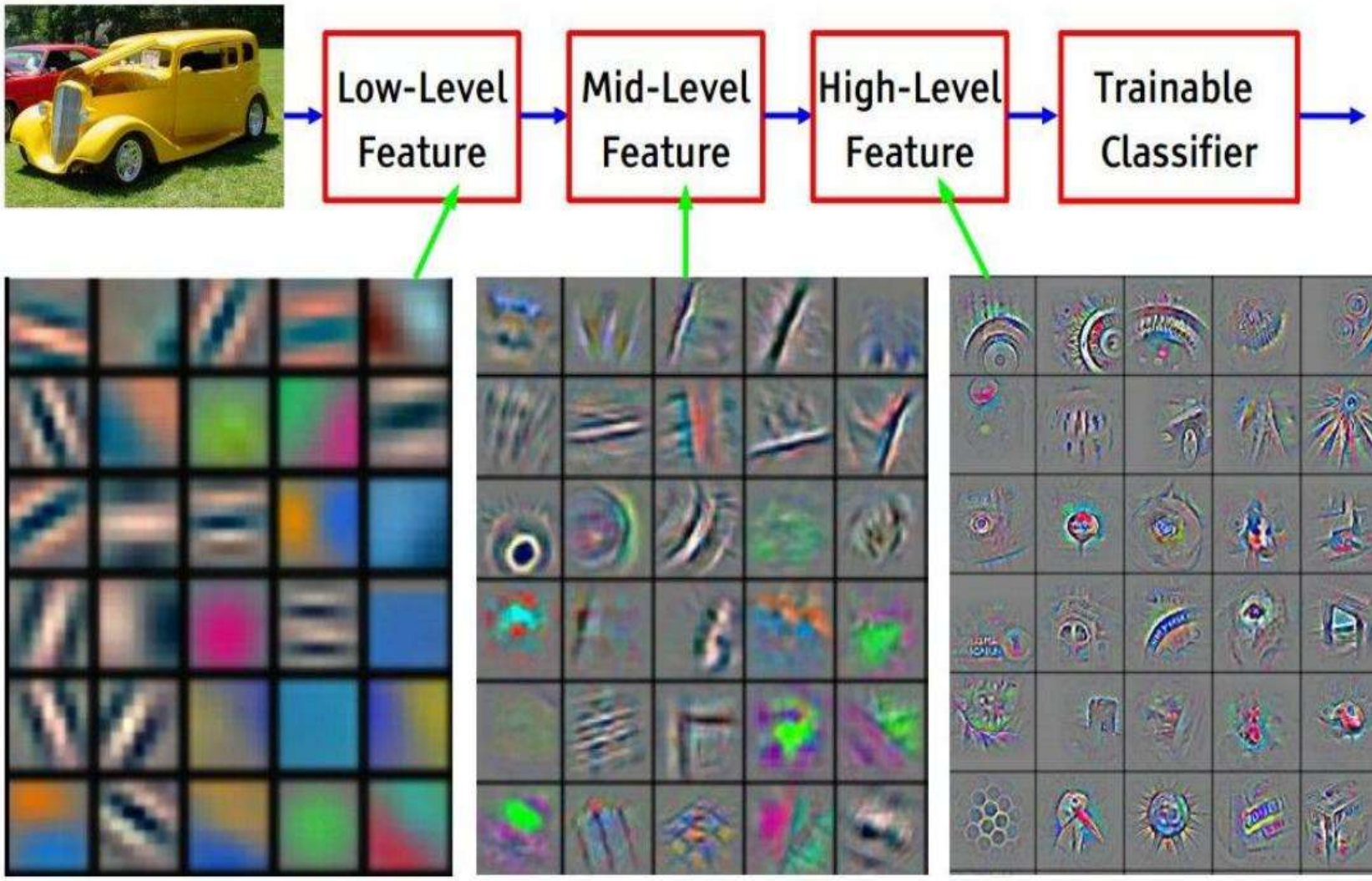
We stack these up to get a "new image" of size 28x28x6!

Preview: ConvNet is a sequence of Convolution Layers, interspersed with activation functions

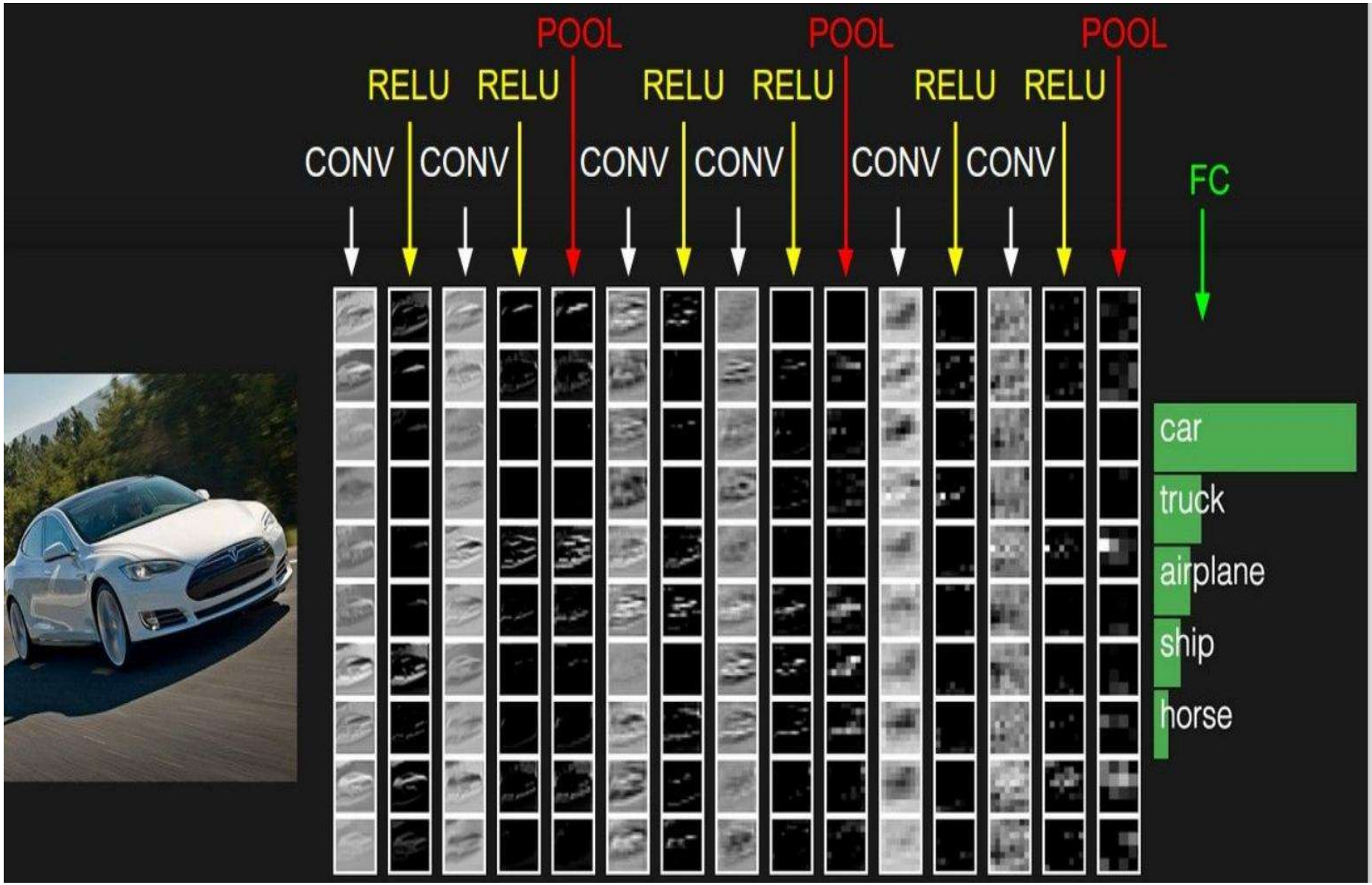


Preview: ConvNet is a sequence of Convolutional Layers, interspersed with activation functions

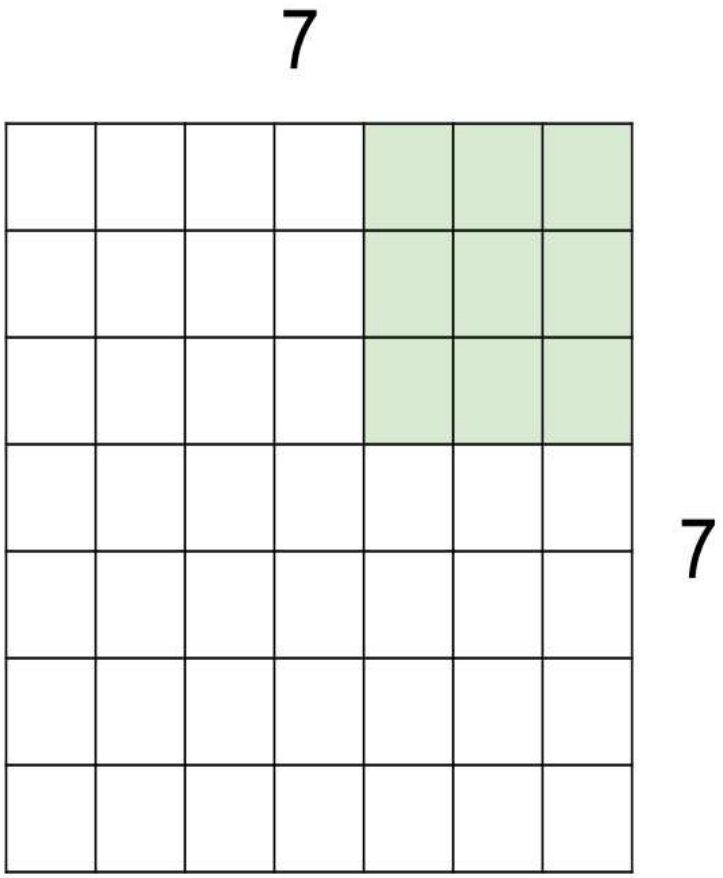




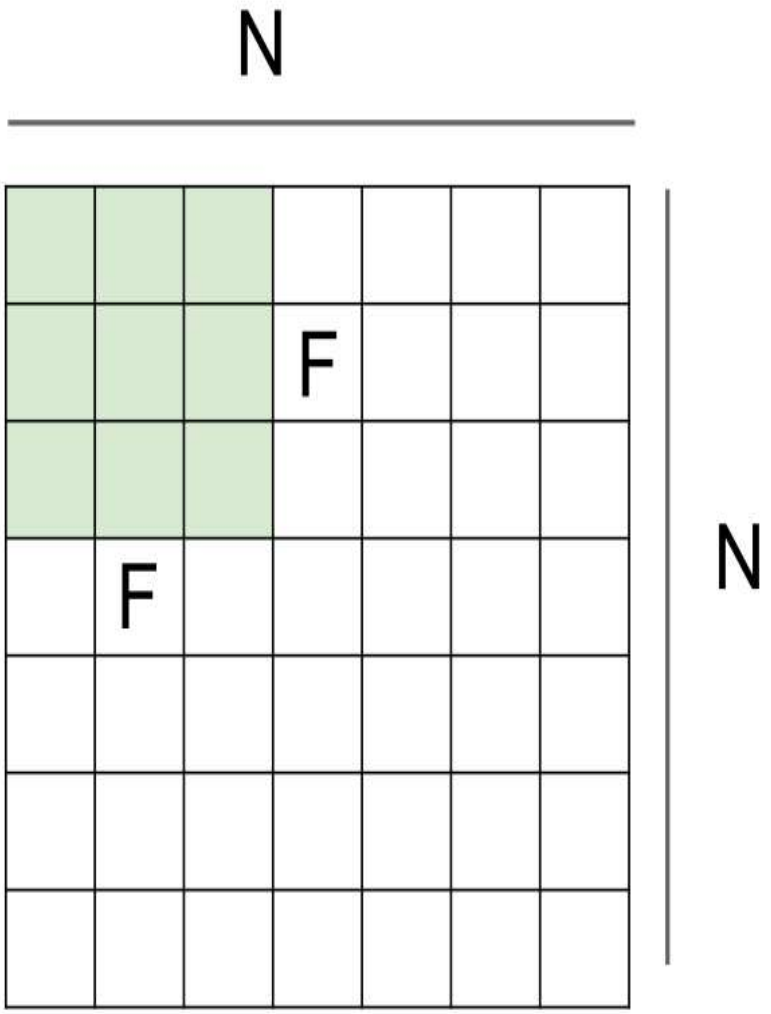
Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]


POOL
POOL
POOL
RELU
RELU
RELU
RELU
RELU
RELU
CONV
CONV
CONV
CONV
CONV
CONV
FC
car
truck
airplane
ship
horse

A closer look at spatial dimensions:



7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!



Output size:

(N - F) / stride + 1

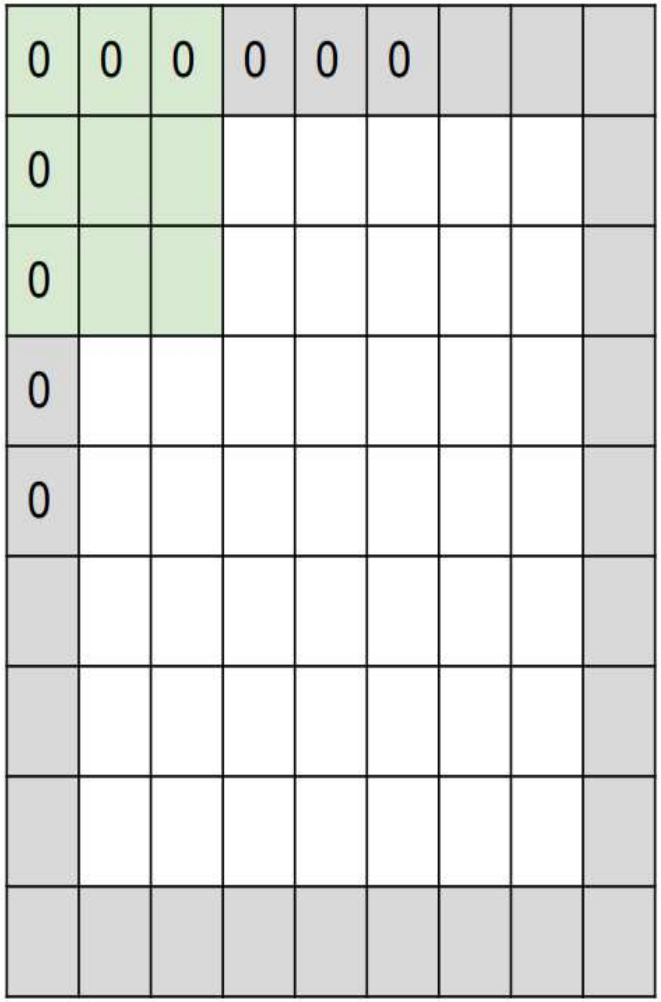
e.g. N = 7, F = 3:

stride 1 => (7 - 3)/1 + 1 = 5

stride 2 => (7 - 3)/2 + 1 = 3

stride 3 => (7 - 3)/3 + 1 = 2.33

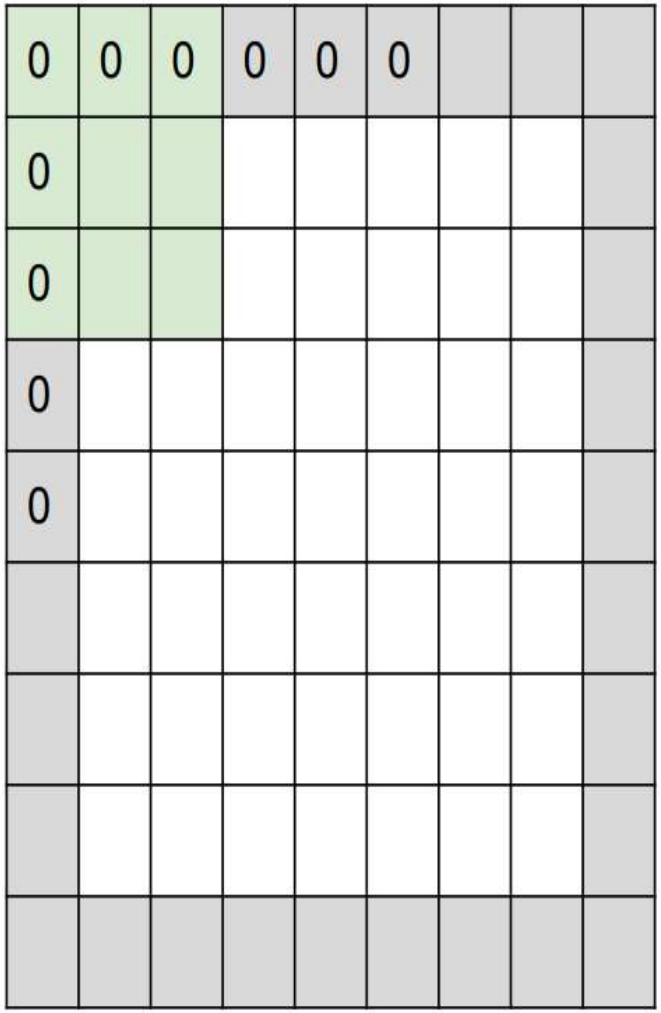
In practice: Common to zero pad the border



e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

(recall:)
(N - F) / stride + 1

In practice: Common to zero pad the border



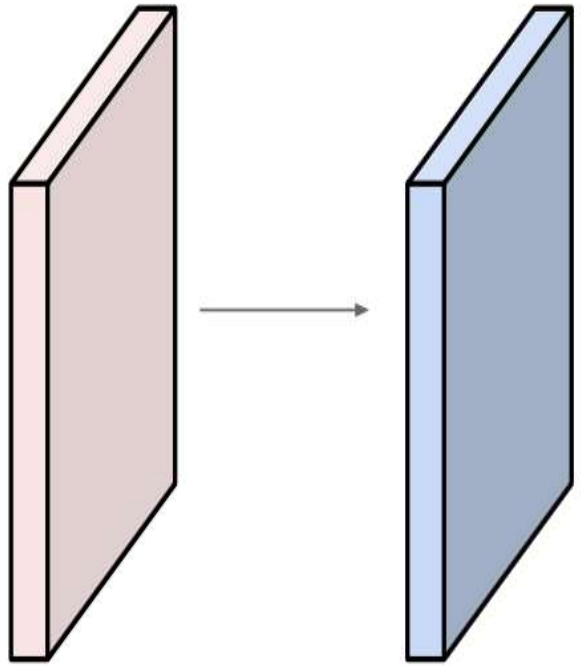
e.g. input 7x7
3x3 filter, applied with **stride 1**
pad with 1 pixel border => what is the output?

7x7 output!

Summary. To summarize, the Conv Layer:

- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
  - Number of filters K,
  - their spatial extent F,
  - the stride S,
  - the amount of zero padding P.
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
  - $W_2 = (W_1 - F + 2P)/S + 1$
  - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. width and height are computed equally by symmetry)
  - $D_2 = K$
- With parameter sharing, it introduces $F \cdot F \cdot D_1$ weights per filter, for a total of $(F \cdot F \cdot D_1) \cdot K$ weights and K biases.
- In the output volume, the d-th depth slice (of size $W_2 \times H_2$) is the result of performing a valid convolution of the d-th filter over the input volume with a stride of S, and then offset by d-th bias.

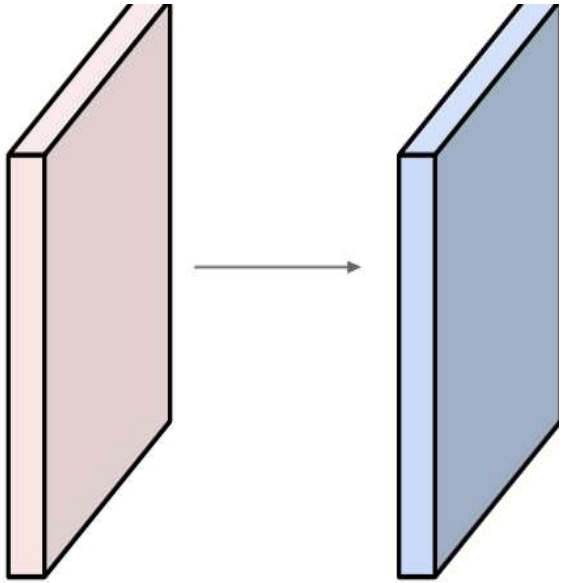
Examples time:



Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Output volume size: ?

Examples time:



Input volume: **32x32x3**
10 5x5 filters with stride 1, pad 2

Output volume size:
(32+2*2-5)/1+1 = 32 spatially, so
32x32x10

Given

- Input image size: **32 × 32 × 3**
- Filter size: **5 × 5**
- Number of filters: **10**
- Stride $S = 1$
- Padding $P = 0$

Compute the output volume

Step 1: Compute spatial size

$$\frac{32 - 5 + 0}{1} + 1 = 28$$

Step 2: Depth

$$= 10 \text{ filters}$$

✅ **Output activation size**

$$\boxed{28 \times 28 \times 10}$$

Given

- Input: **64 × 64 × 3**
- Kernel: **3 × 3**
- Filters: **32**
- Stride $S = 1$
- Padding $P = 1$

Compute the output volume

$$\frac{64 - 3 + 2(1)}{1} + 1 = 64$$

Output size

$$\boxed{64 \times 64 \times 32}$$

The output size after a max pooling layer is computed as $\left\lfloor \frac{N-F+2P}{S} \right\rfloor + 1$, where N is input size, F is pool size, S is stride, and P is padding.

Given

- Input activation: **28 × 28 × 16**
- Pool size: **2 × 2**
- Stride $S = 2$
- Padding $P = 0$

Compute the output volume

The output size after a max pooling layer is computed as $\left\lfloor \frac{N-F+2P}{S} \right\rfloor + 1$, where N is input size, F is pool size, S is stride, and P is padding.

Given

- Input activation: **28 × 28 × 16**
- Pool size: **2 × 2**
- Stride $S = 2$
- Padding $P = 0$

Solution

$$\frac{28 - 2}{2} + 1 = 14$$

✅ **Output size**

$$\boxed{14 \times 14 \times 16}$$

Given CNN Architecture

```
Input: 32 × 32 × 3
↓ Conv1: 3×3, 8 filters, stride 1, padding 1
↓ MaxPool: 2×2, stride 2
↓ Conv2: 5×5, 16 filters, stride 1, padding 0
```

Compute the output volume

Step-1

- **Conv1**

$$\frac{32-3+2}{1}+1=32$$

Output:

$$32 \times 32 \times 8$$

Step-2

- **Max Pool**

$$\frac{32-2}{2}+1=16$$

Output:

$$16 \times 16 \times 8$$

Step-3

- **Conv2**

$$\frac{16 - 5 + 0}{1} + 1 = 12$$

Depth = 16 filters

Step-4

✅ **Final Output**

$$\boxed{12 \times 12 \times 16}$$

Global Max Pooling takes the **maximum value over the entire spatial dimension** of each feature map (channel).

Input activation:

$$H \times W \times C$$

Output after Global Max Pooling:

$$\boxed{1 \times 1 \times C}$$

That is:

- Height → 1
- Width → 1
- Channels → unchanged

Why use Global Max Pooling?

Advantages

- **No parameters** (unlike fully connected layers)
- **Reduces overfitting**
- **Translation invariant**
- Highlights **strongest feature presence**
- Reduces model size significantly

Global Max Pooling vs Max Pooling

Aspect	Max Pooling	Global Max Pooling
Window size	Fixed (e.g., 2×2)	Entire feature map
Output size	Reduced H × W	1 × 1
Parameters	None	None
Usage	Intermediate layers	Near output layer

Input activation size:

$$4 \times 4 \times 3$$

Channel 1:

$$\begin{bmatrix} 1 & 2 & 0 & 1 \\ 3 & 1 & 2 & 2 \\ 0 & 1 & 1 & 0 \\ 2 & 1 & 3 & 1 \end{bmatrix}$$

Maximum = **3**

Channel 2:

$$\begin{bmatrix} 0 & 1 & 1 & 0 \\ 2 & 3 & 1 & 1 \\ 1 & 0 & 2 & 1 \\ 1 & 1 & 0 & 2 \end{bmatrix}$$

Maximum = **3**

Channel 3:

$$\begin{bmatrix} 1 & 0 & 2 & 1 \\ 0 & 1 & 1 & 3 \\ 2 & 1 & 0 & 1 \\ 1 & 2 & 1 & 0 \end{bmatrix}$$

Maximum = **3**

✅ **Output after Global Max Pooling:**

$$[3,\ 3,\ 3]$$

Output size:

$$1 \times 1 \times 3$$